

CSCE 611

Advanced Digital Design

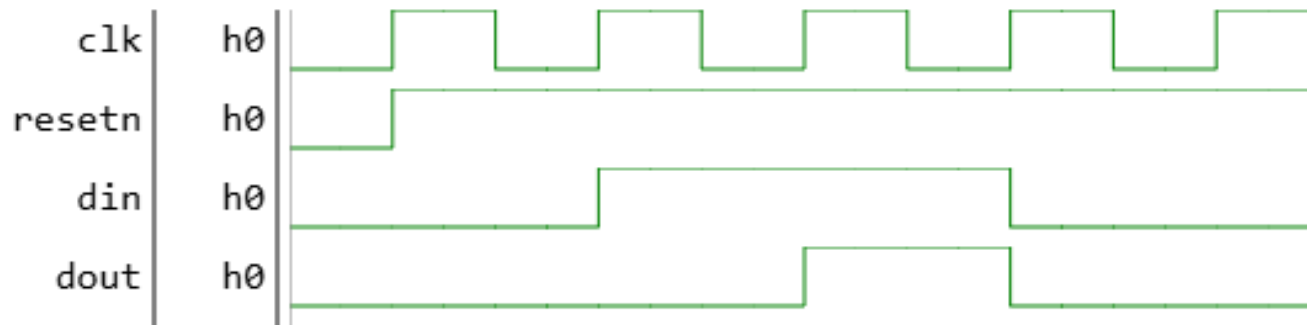


RISC-V Microarchitecture



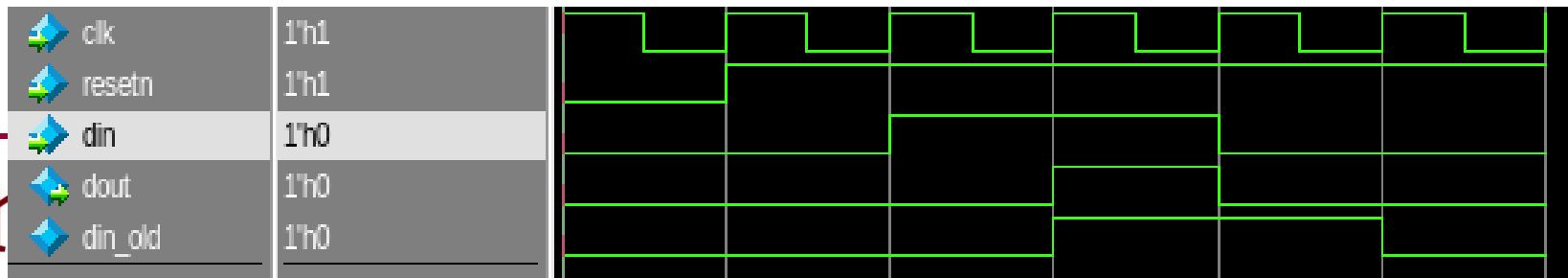
Example 1 (from chipdev.io)

- Build a circuit that pulses dout one cycle after the rising edge of din. A pulse is defined as writing a single-cycle 1 as shown below. When resetn is asserted, the value of din should be treated as 0.



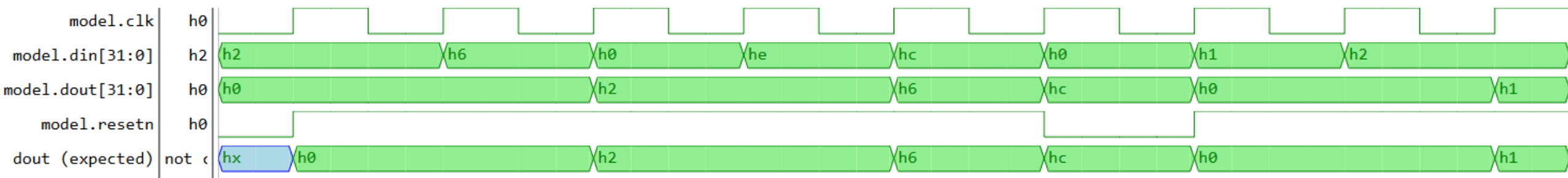
Example 1 (from chipdev.io)

```
module model (  
    input clk,  
    input resetn,  
    input din,  
    output logic dout  
);  
  
logic din_old;  
  
    always @(posedge clk) begin  
        if (~resetn) begin  
            dout <= 1'b0;  
            din_old <= 1'b0;  
        end else begin  
            din_old <= din;  
            if (din_old == 1'b0 && din == 1'b1)  
                dout <= 1'b1;  
            else  
                dout <= 1'b0;  
        end  
    end  
end  
  
endmodule
```



Example 2 (from chipdev.io)

- Given a clocked sequence of unsigned values, output the second-largest value seen so far in the sequence. If only one value is seen, then the output (dout) should equal 0. Note that repeated values are treated as separate candidates for being the second largest value. When the reset-low signal (resetn) goes low, all previous values seen in the input sequence should no longer be considered for the calculation of the second largest value, and the output dout should restart from 0 on the next cycle.

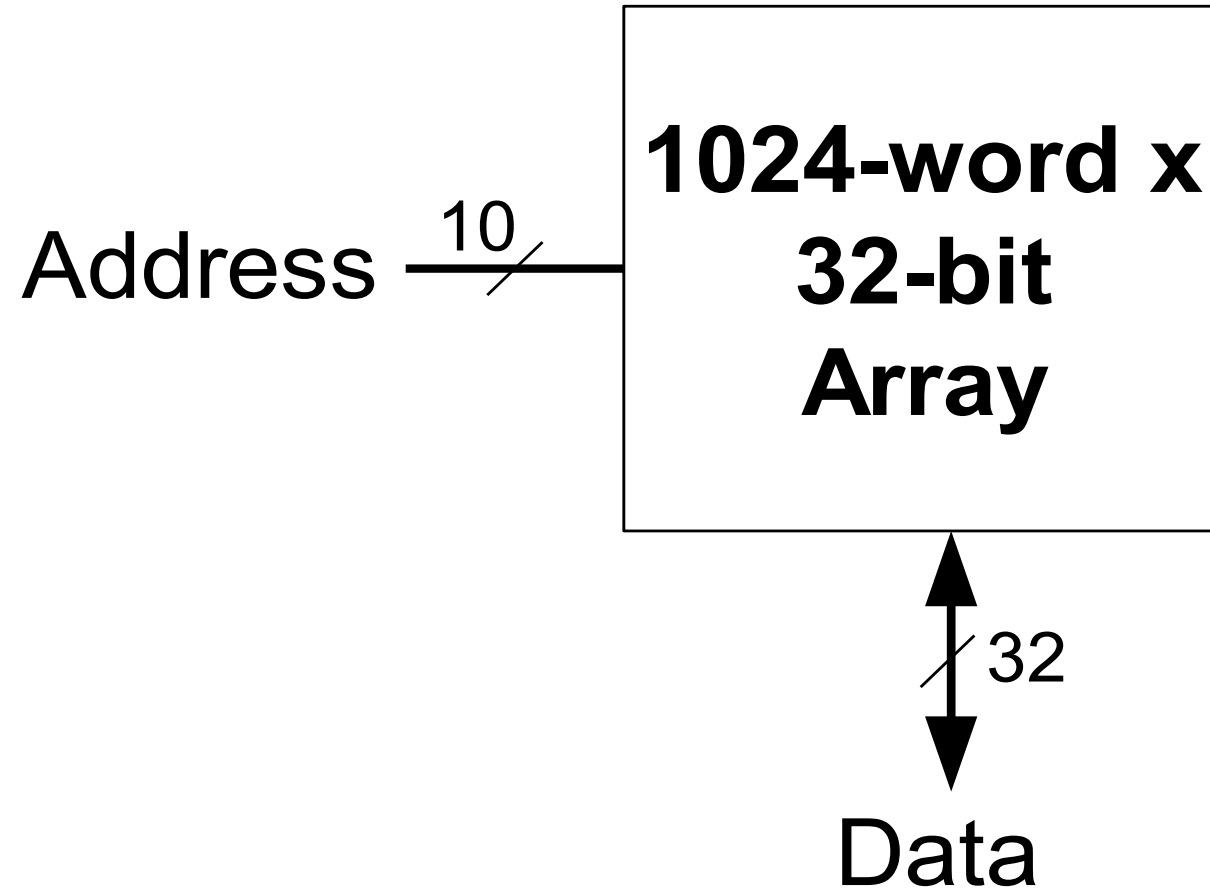


Example 2 (from chipdev.io)

```
module model (  
    input clk,  
    input resetn,  
    input [31:0] din,  
    output logic [31:0] dout  
);  
  
logic [DATA_WIDTH-  
1:0] largest, second_largest  
= 'h0;
```

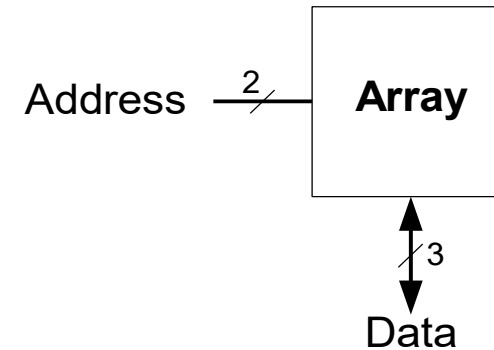
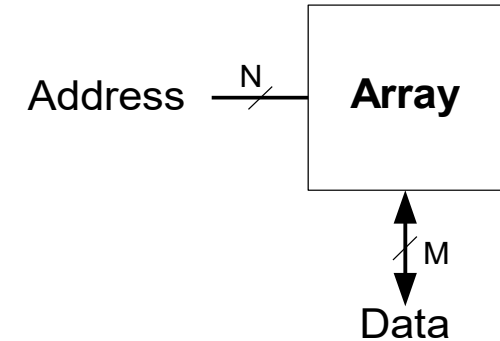
```
    always_ff @(posedge clk) begin  
        if (~resetn) begin  
            largest <= 'h0;  
            second_largest <= 'h0;  
        end else if (din > largest) begin  
            largest <= din;  
            second_largest <= largest;  
        end else if (din > second_largest) begin  
            second_largest <= din;  
        end  
    end  
  
    assign dout = second_largest;  
  
endmodule
```

Memory Arrays



Memory Arrays

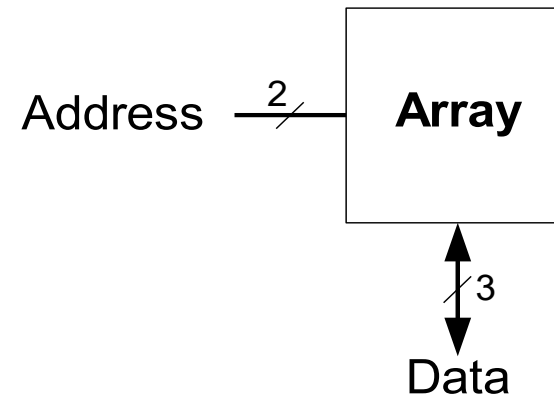
- 2-dimensional array of bit cells
- Each bit cell stores one bit
- N address bits and M data bits:
 - 2^N rows and M columns
 - **Depth:** number of rows (number of words)
 - **Width:** number of columns (size of word)
 - **Array size:** $\text{depth} \times \text{width} = 2^N \times M$



Address	Data			
11	0	1	0	depth ↑ ↓
10	1	0	0	
01	1	1	0	
00	0	1	1	
	width ← →			

Memory Array Example

- $2^2 \times 3$ -bit array
- Number of words: 4
- Word size: 3-bits
- For example, the 3-bit word stored at address 10 is 100



Address	Data			
11	0	1	0	depth ↑ ↓
10	1	0	0	
01	1	1	0	
00	0	1	1	
	width ←→			

RAM

- 8192x32 RAM (asynchronous read):

```
logic [31:0] mem[8191:0];  
logic [12:0] addr;  
logic [31:0] readdata,writedata;  
logic we,clk;
```

```
initial $readmemh("mem.txt",mem);
```

```
assign readdata = mem[addr];  
always_ff @(posedge clk)  
    if (we) mem[addr] <= writedata;
```

step 1: declare the RAM with desired geometry

step 2: initialize RAM (optional)

step 3: add read ports
(assign/always_comb for asynchronous,
always_ff for synchronous)

step 4: add write ports
(must use always_ff)

RAM

- 8192x32 RAM (synchronous read):

```
logic [31:0] mem[8191:0];  
logic [12:0] addr;  
logic [31:0] readdata,writedata;  
logic we,clk;  
  
initial $readmemh("mem.txt",mem);  
  
always_ff @(posedge clk) begin  
    if (we) mem[addr] <= writedata;  
    readdata <= mem[addr];  
end
```

Asynchronous vs Synchronous Read

- Asynchronous:

addr	0004	0004		0005		0006					
we	0										
clk	1										
readdata	00000004	00000004		00000005		00000006		deadbeef			
writedata	00000000	00000000				deadbeef		00000000			

- Synchronous:

addr	0004	0004		0005		0006					
we	0										
clk	1										
readdata	xxxxxxxx		00000004		00000005		00000006		deadbeef		
writedata	00000000	00000000			deadbeef		00000000				

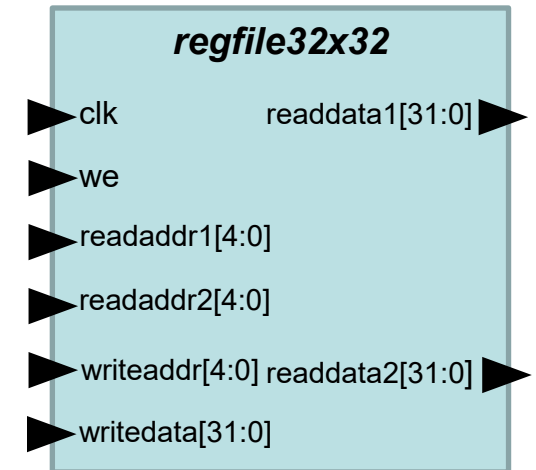
Register File

```
module regfile32x32 (input logic we,clk,
                    input logic [4:0] readaddr1,
                        readaddr2,
                        writeaddr,
                    output logic [31:0] readdata1,readdata2,
                    input logic [31:0] writedata);

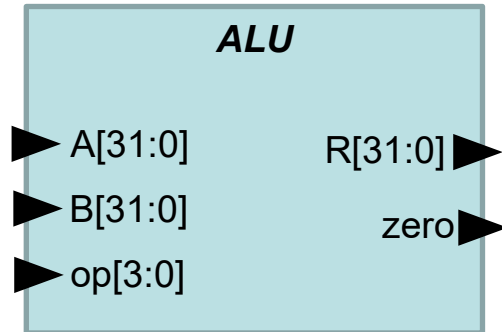
logic [31:0] mem[31:0];
assign readdata1 = readaddr1 == 5'b0 ? 32'b0 :
                readaddr1==writeaddr && we ? writedata :
                mem[readaddr1];

assign readdata2 = readaddr2 == 5'b0 ? 32'b0 :
                readaddr2==writeaddr && we ? writedata :
                mem[readaddr2];

always_ff @(posedge clk)
    if (we) mem[writeaddr] <= writedata;
endmodule
```



Arithmetic Logic Unit (ALU)



- 4 bit multiply:
 - $0001 * 1111 = 0000\ 1111$ (unsigned)
 - $0001 * 1111 = 1111\ 1111$ (signed)
- 4 bit SLT:
 - $0001 < 1111 = \text{TRUE}$ (unsigned)
 - $0001 < 1111 = \text{FALSE}$ (signed)

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

ALU Design

```
module alu(  
    input logic [31:0] A,  
    input logic [31:0] B,  
    input logic [ 3:0] op,  
    output logic [31:0] R,  
    output logic zero  
);  
  
logic [31:0] mulls, mullu, mulhu, mulhs;  
logic [ 4:0] shamt;  
logic [31:0] shifted;  
  
assign zero = R==32'd0 ? 1'b1 : 1'b0;  
  
assign {mulhu, mullu} = A*B;  
assign {mulhs, mulls} = $signed(A)*$signed(B);  
assign shamt = B[4:0];  
  
// Arithmetic right shift does not work under ModelSim, so we we work  
// around this by implementing our own shift. Notice the blocking assignments.  
always_comb begin  
    shifted = A;  
    if (shamt[0]) shifted = {{1{A[31]}},shifted[31:1]};  
    if (shamt[1]) shifted = {{2{A[31]}},shifted[31:2]};  
    if (shamt[2]) shifted = {{4{A[31]}},shifted[31:4]};  
    if (shamt[3]) shifted = {{8{A[31]}},shifted[31:8]};  
    if (shamt[4]) shifted = {{16{A[31]}},shifted[31:16]};  
  
end  
  
assign R =  
    (op == 4'b0000) ? A & B :  
    (op == 4'b0001) ? A | B :  
    (op == 4'b0010) ? A ^ B :  
    (op == 4'b0011) ? A + B :  
    (op == 4'b0100) ? A - B :  
    (op == 4'b0101) ? mulls :  
    (op == 4'b0110) ? mulhs :  
    (op == 4'b0111) ? mulhu :  
    (op == 4'b1000) ? A << shamt :  
    (op == 4'b1001) ? A >> shamt :  
    (op == 4'b1010) ? shifted :  
    (op == 4'b1011) ? shifted :  
    (op == 4'b1100) ? ($signed(A) < $signed(B)) :  
    (op == 4'b1101) ? (A < B) :  
    (op == 4'b1110) ? (A < B) : (A < B);  
  
endmodule
```

22 RISC-V Instructions (Lab 3)

- Goal: design a pipelined processor that can execute a minimal set of RISC-V instructions:
 - Arithmetic R-type: `add, sub, mul, mulh, mulhu`
 - Arithmetic I-type: `addi`
 - Comparison R-type: `slt, sltu`
 - Logical R-type: `and, or, xor`
 - Logical I-type: `andi, ori, xori`
 - Shift R-type: `sll, srl, sra`
 - Shift I-type: `slli, srai, srli`
 - U-type: `lui`
 - I/O I-type: `csrrw`

RISC-V Instruction Formats

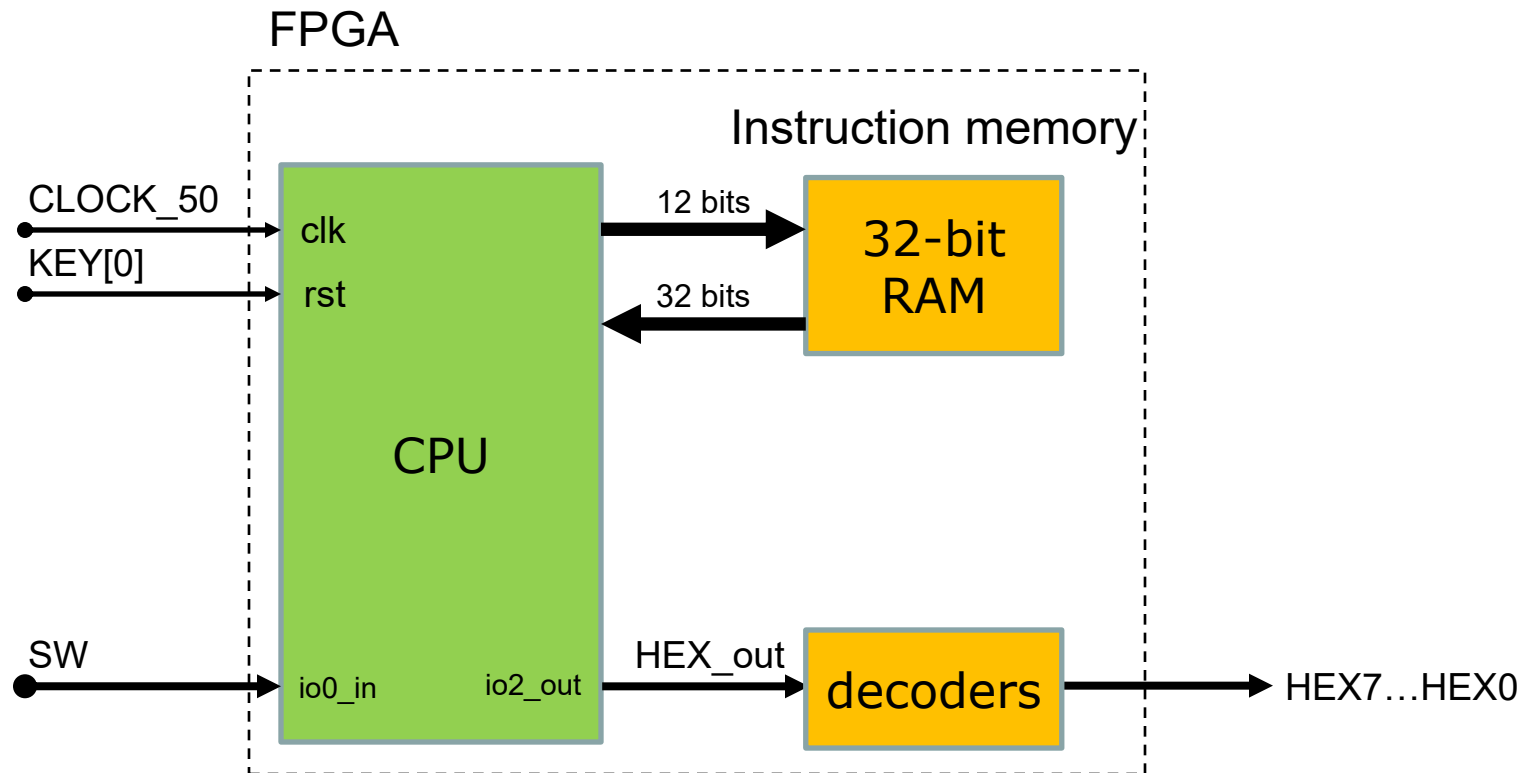
R-type	31:25 (7 bits)	24:20 (5 bits)	19:15 (5 bits)	14:12 (3 bits)	11:7 (5 bits)	6:0 (7 bits)	add rd,rs1,rs2 and rd,rs1,rs2 sub rd,rs1,rs2 or rd,rs1,rs2 mul rd,rs1,rs2 xor rd,rs1,rs2 mulh rd,rs1,rs2 sll rd,rs1,rs2 mulhu rd,rs1,rs2 srl rd,rs1,rs2 slt rd,rs1,rs2 sra rd,rs1,rs2 sltu rd,rs1,rs2
	funct7	rs2	rs1	funct3	rd	opcode	
I-type	31:20 (12 bits)		19:15 (5 bits)	14:12 (3 bits)	11:7 (5 bits)	6:0 (7 bits)	addi rd,rs1,imm slli rd,rs1,imm andi rd,rs1,imm srli rd,rs1,imm ori rd,rs1,imm srli rd,rs1,imm xori rd,rs1,imm csrrw rd,imm,rs1
	imm12[11:0]		rs1	funct3	rd	opcode	
U-type	31:12 (20 bits)				11:7 (5 bits)	6:0 (7 bits)	lui rd,imm
	imm20[31:12]				rd	opcode	

csrrw Instruction

- `csrrw rd,csr,rs1`
 - "Control and status register read and write"
 - Concurrently update `CSR[imm12]` and place old value of `CSR[imm12]` into `rd`
 - `t = CSRs[imm12];`
 - `CSRs[imm12] = R[rs1];`
 - `R[rd] = t;`
- For us:
 - `csrrw rd,0xf00,x0:` write the state of the switches into `rd`
 - `csrrw x0,0xf02,x1:` write the value in `x1` to the HEX displays

Top-Level Design

- Use on-chip memory for program
 - Word addressed



Getting Started on Lab 3

```
module cpu(input logic clk, input logic rst_n);

    logic [31:0] inst_ram [4095:0];
    initial $readmemh("program.rom",inst_ram);

    logic [11:0] PC_FETCH = 12'd0;
    logic [31:0] instruction_EX;

    always_ff @(posedge clk) begin
        if (~rst_n) begin
            PC_FETCH <= 12'd0;
            instruction_EX <= 32'd0;
        end else begin
            PC_FETCH <= PC_FETCH + 1'b1;
            instruction_EX <= inst_ram[PC_FETCH];
        end
    end

end

endmodule
```

Getting Started on Lab 3

- in top.sv:

```
cpu mycpu(.clk(CLOCK_50),.rst_n(KEY[0]));
```

- in simtop.sv:

```
top dut
(
    //////////// CLOCK ////////////
    .CLOCK_50(clk),
    .CLOCK2_50(),
    .CLOCK3_50(),

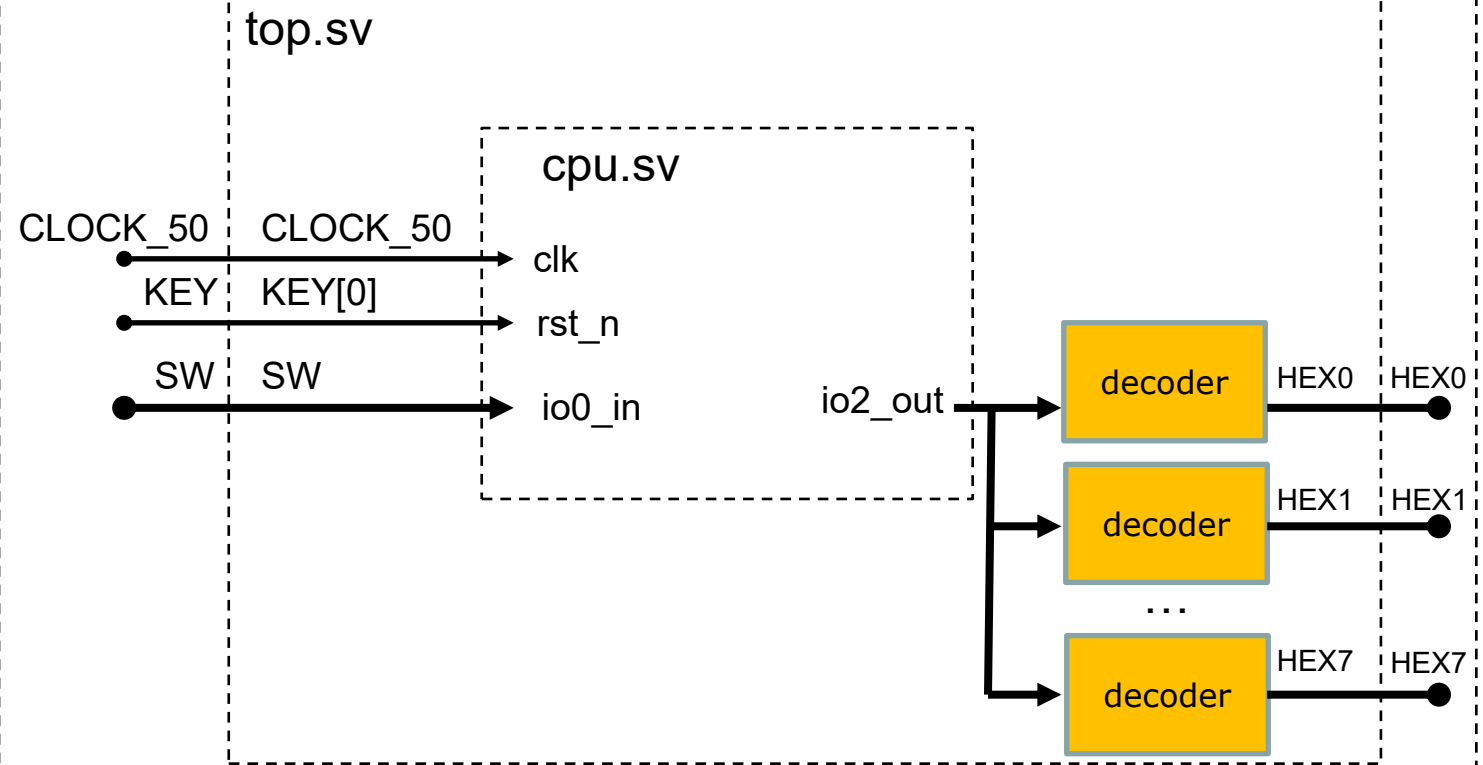
    //////////// LED ////////////
    .LEDG(),
    .LEDR(),

    //////////// KEY ////////////
    .KEY(key),

    //////////// SW ////////////
    .SW(SW),

    //////////// SEG7 ////////////
    .HEX0(HEX0),
    .HEX1(HEX1),
    .HEX2(HEX2),
    .HEX3(HEX3),
    .HEX4(HEX4),
    .HEX5(HEX5),
    .HEX6(HEX6),
    .HEX7(HEX7)
);
```

simtop.sv

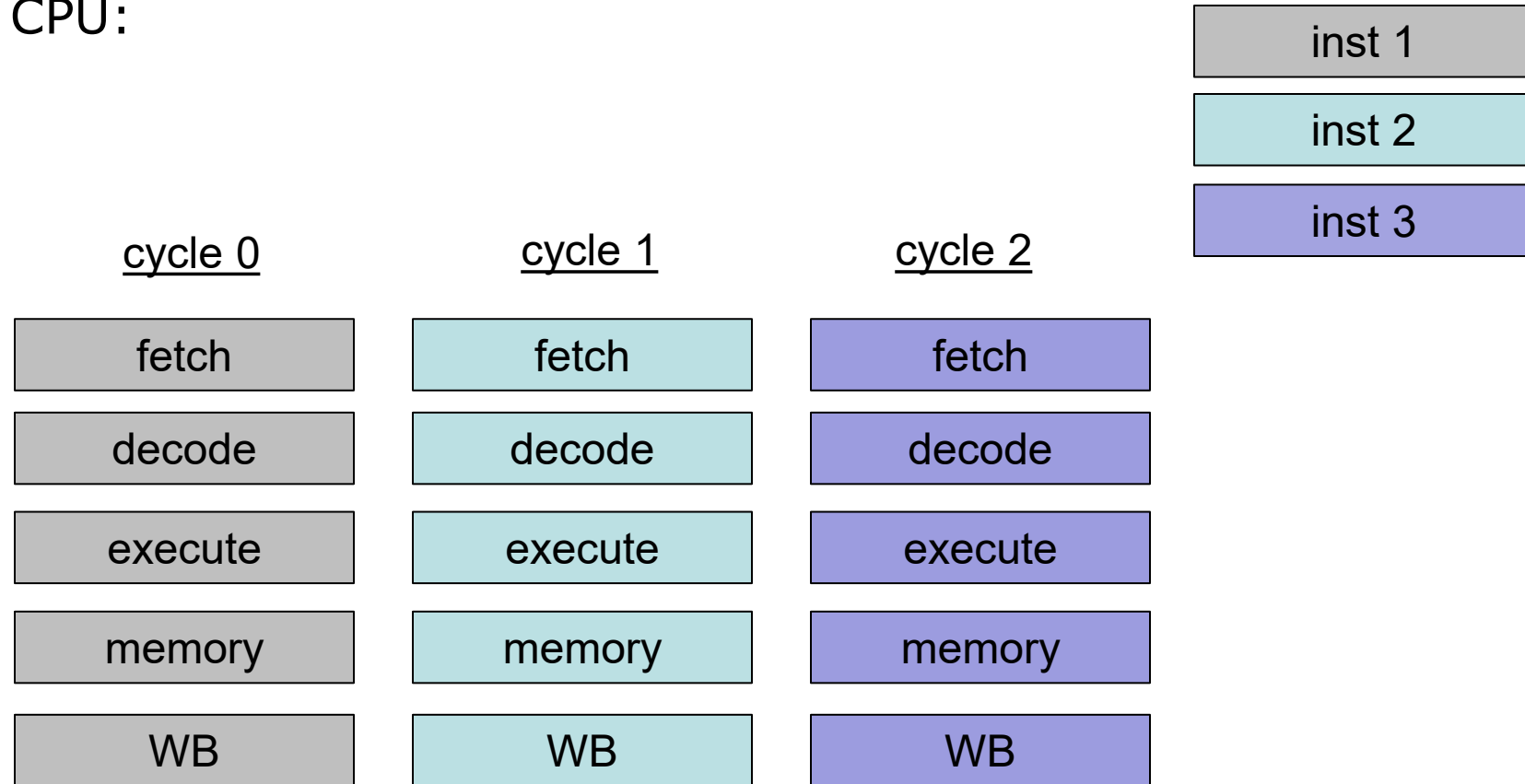


Execution Stages

- Recall basic steps: fetch, decode, execute, memory, write back
 - R-type computational instructions (ex. ADD x1, x2, x3):
 - fetch, decode, execute, write back
 - Branch instructions
 - fetch, decode, execute
 - Load instruction
 - fetch, decode, execute, memory, write back
 - Store instruction
 - fetch, decode, memory

Execution Stages

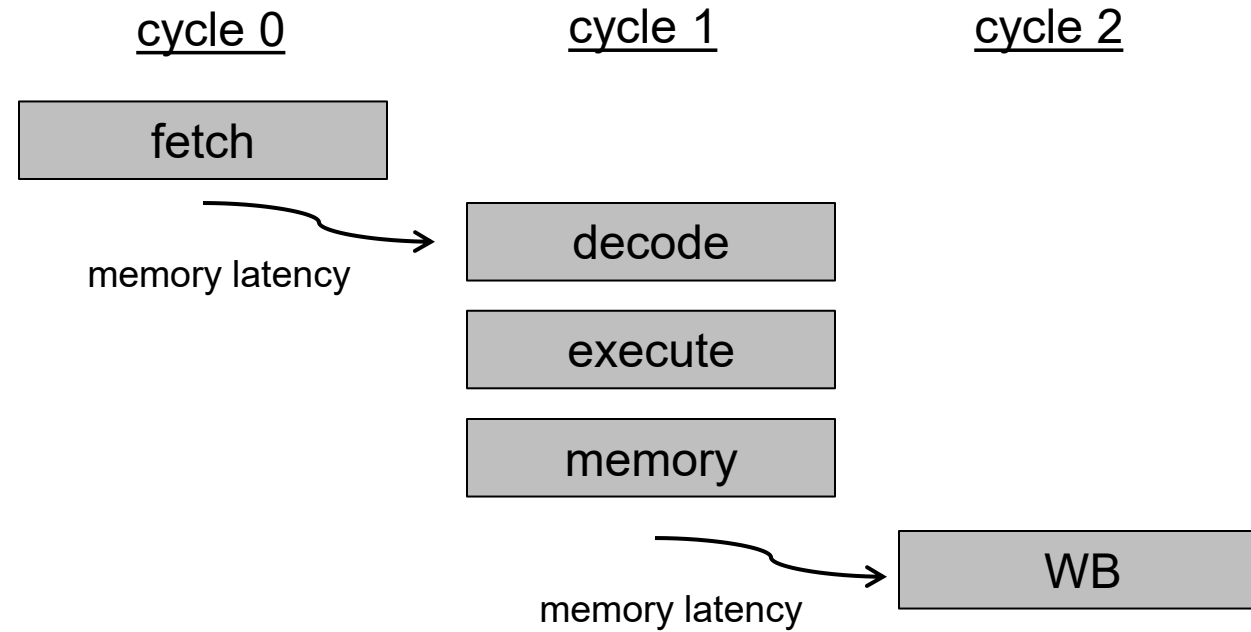
- Single cycle CPU:



CPU Design

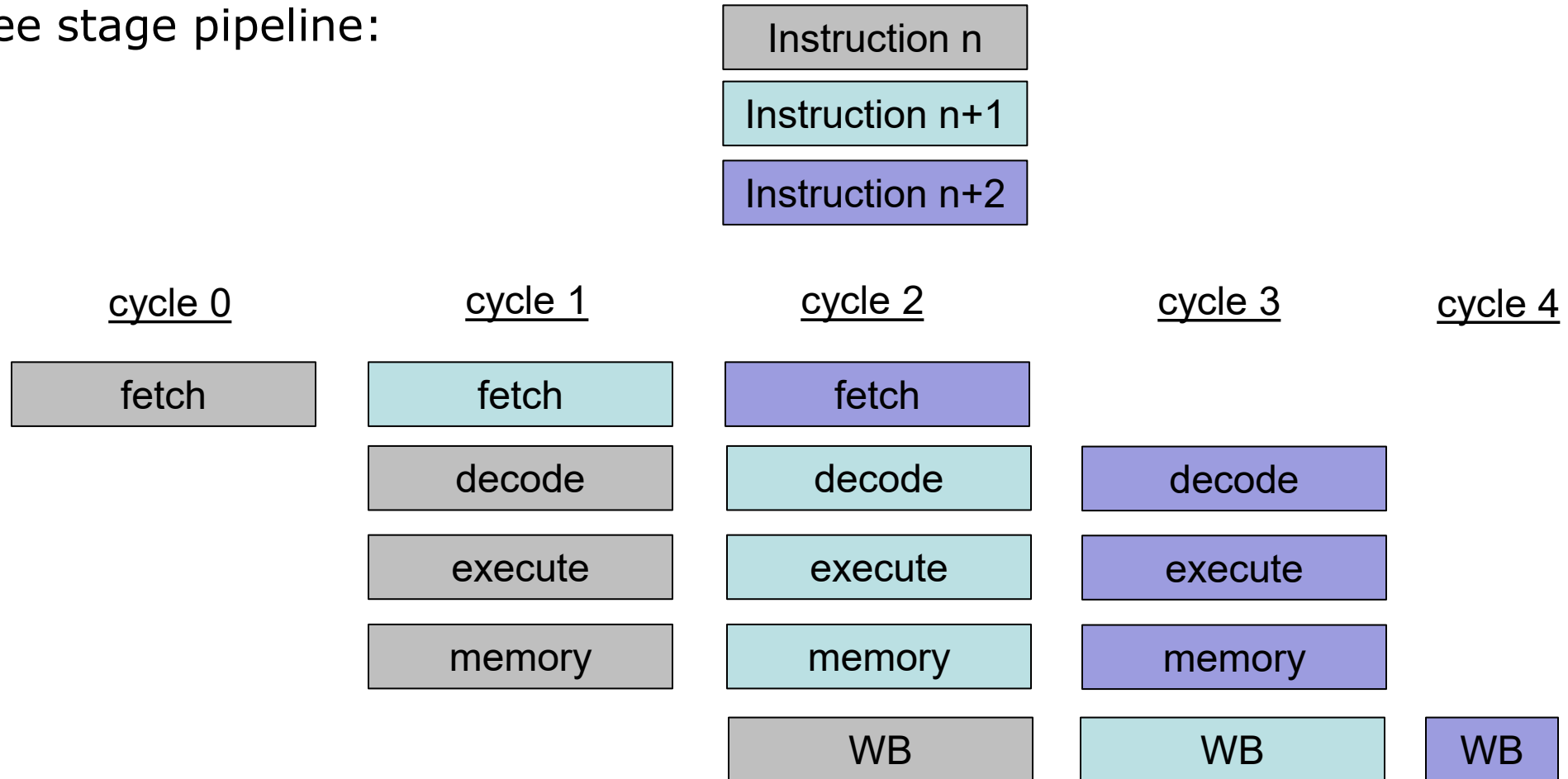
- Loads have a one-cycle latency
 - Need to separate MEM and WB
- Fetches have a one-cycle latency
 - Need to separate FETCH and DECODE
- Solution:
 - Three-stage pipeline:
 - FETCH, DECODE/EX/MEM, WB
 - F E W
 - Add pipeline registers between E and W
 - Loaded data is already delayed, so don't need a register for this
 - Leads to control hazard
 - Must flush FETCH for taken branches
 - Zero-out control signals in if branch is taken in previous cycle

Execution Stages

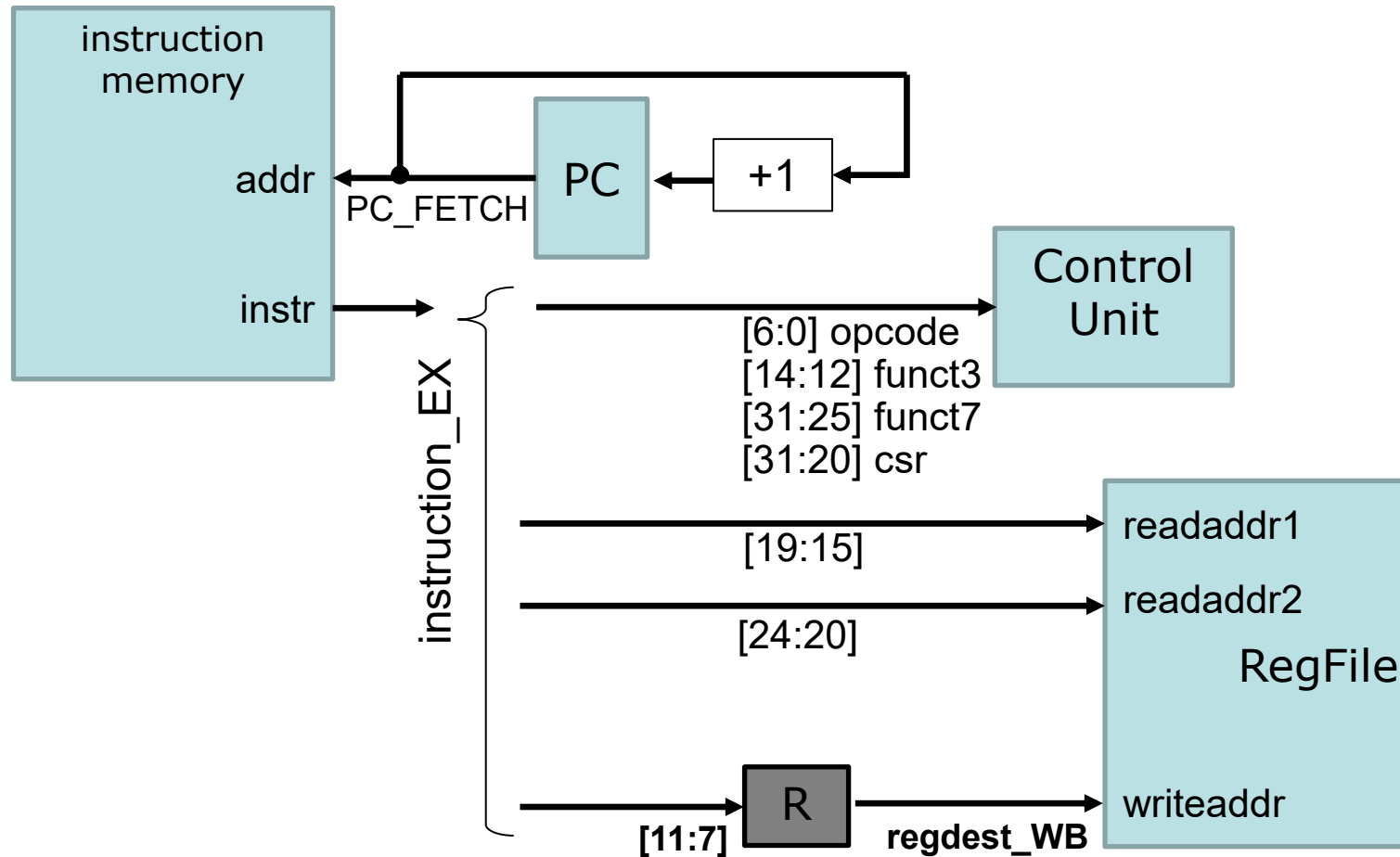


Execution Stages

- Three stage pipeline:

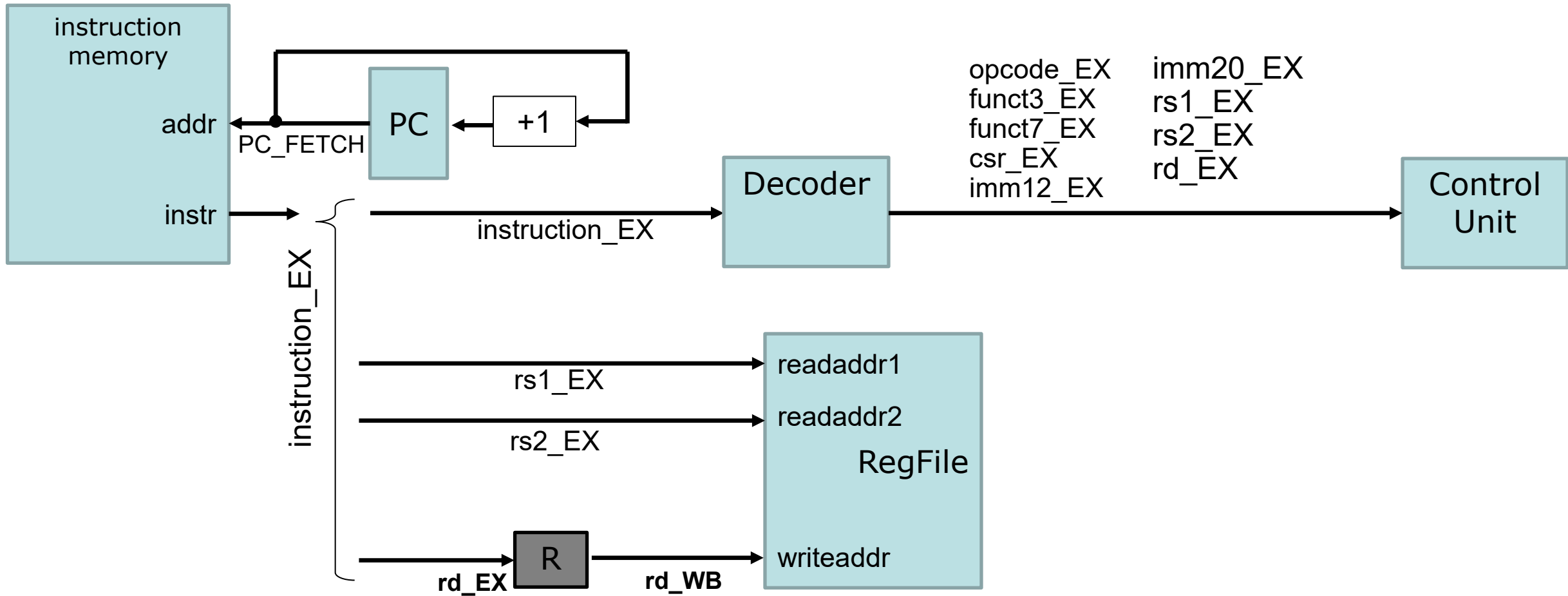


RISC-V Fetch/Decode



- Instruction bits:
 - To control unit:
 - To register file:
 - [24:20] rs1 address (readaddr1)
 - [19:15] rs2 address (readaddr2)

RISC-V Fetch/Decode



Initializing Instruction Memory

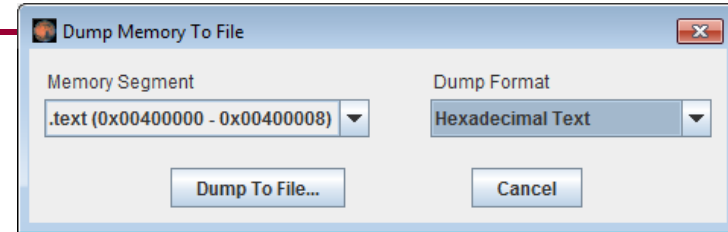
- Use script:

- Use RARS to assemble (F3)
- File | Dump Memory
- Use this file to initialize instruction RAM:

```
logic [31:0] instruction_mem [4095:0];  
logic [31:0] instruction_EX;  
initial $readmemh("hexcode.txt", instruction_mem);
```

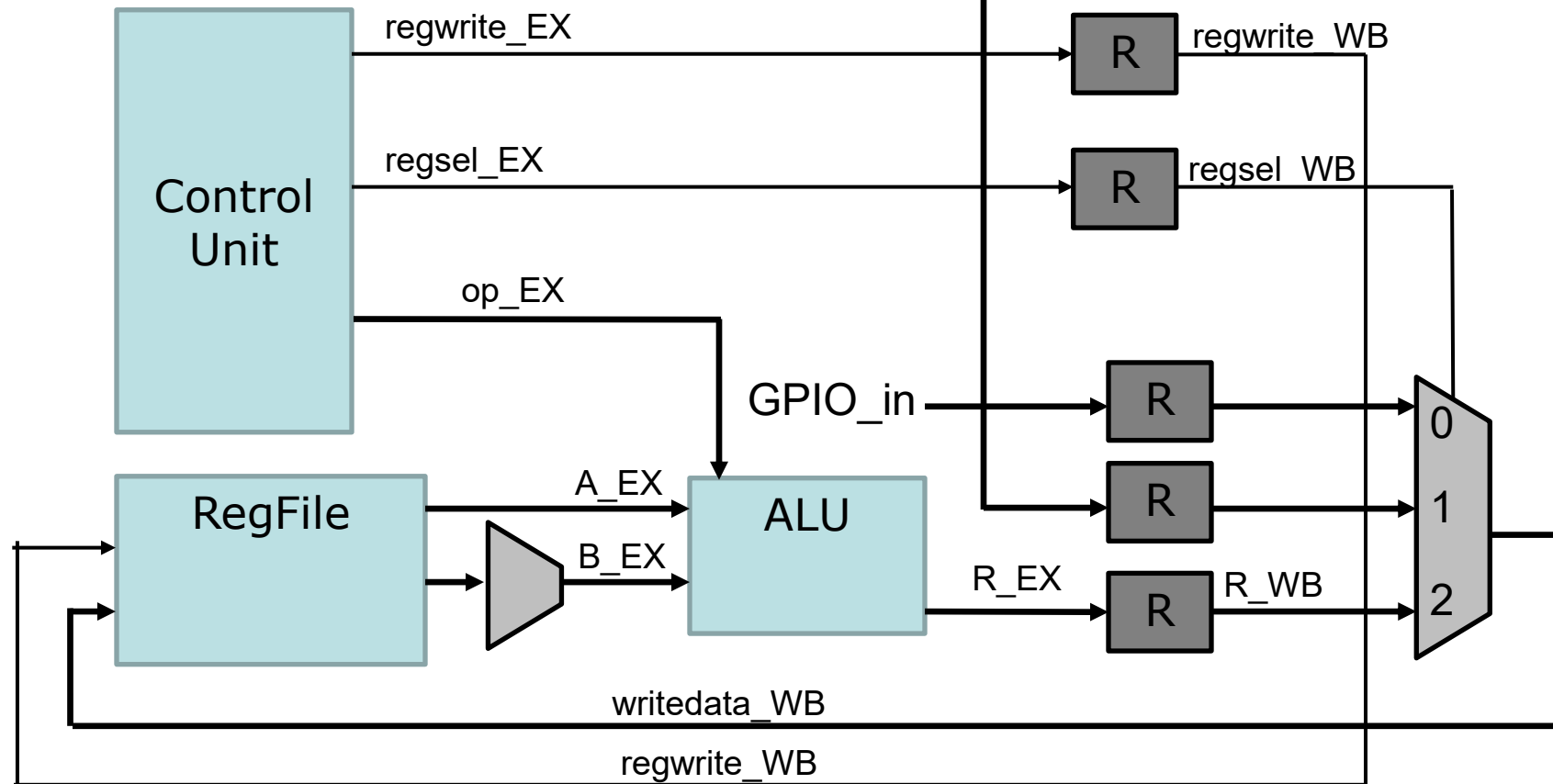
- Implementation:

```
always_ff @(posedge clk)  
if (rst) begin  
    instruction_EX <= 32'b0;  
    PC_FETCH <= 12'b0;  
end else begin  
    instruction_EX <= instruction_mem[PC_FETCH];  
    PC_FETCH <= PC_FETCH + 12'b1;  
end
```

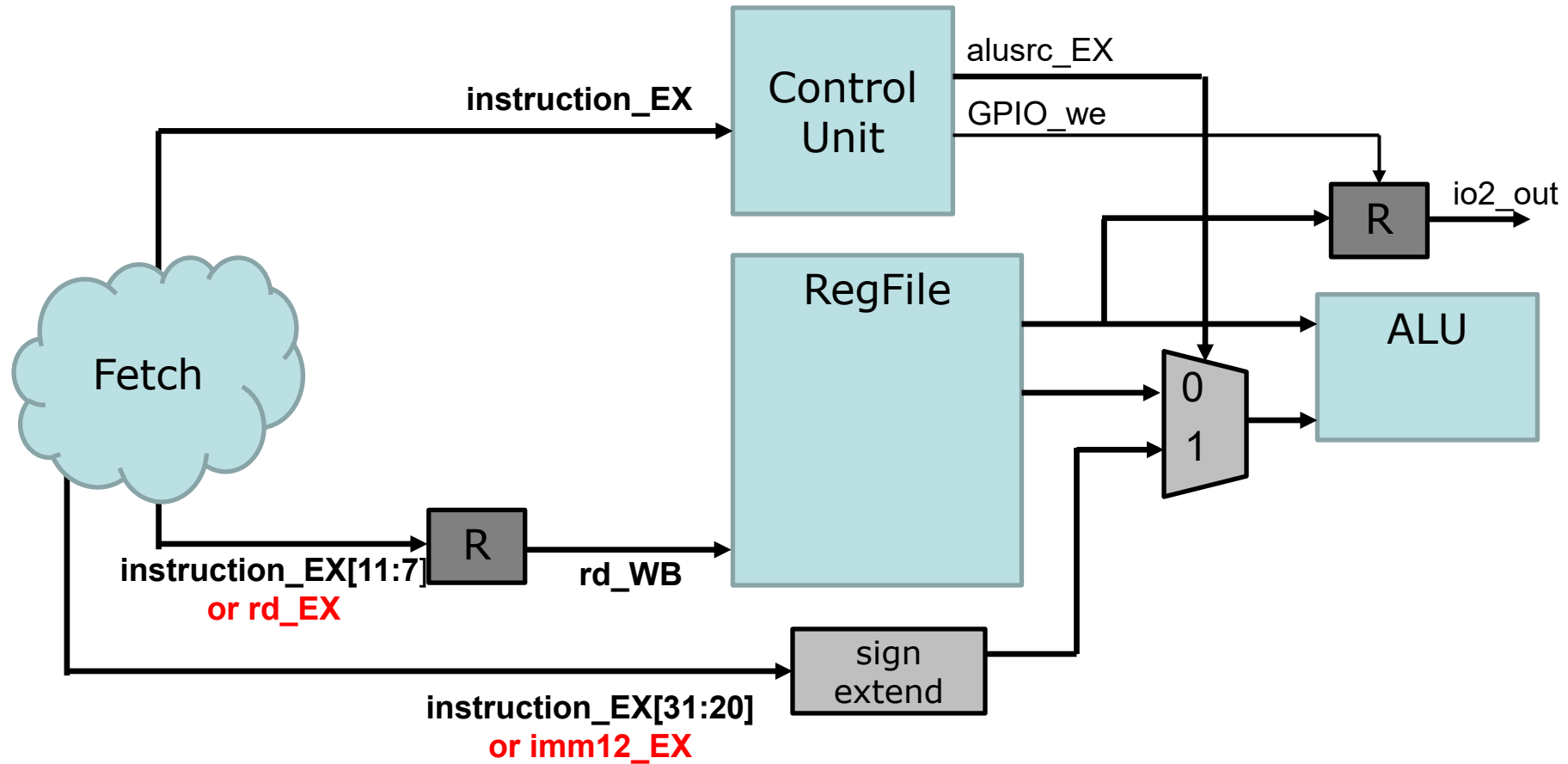


EX/WB Stage

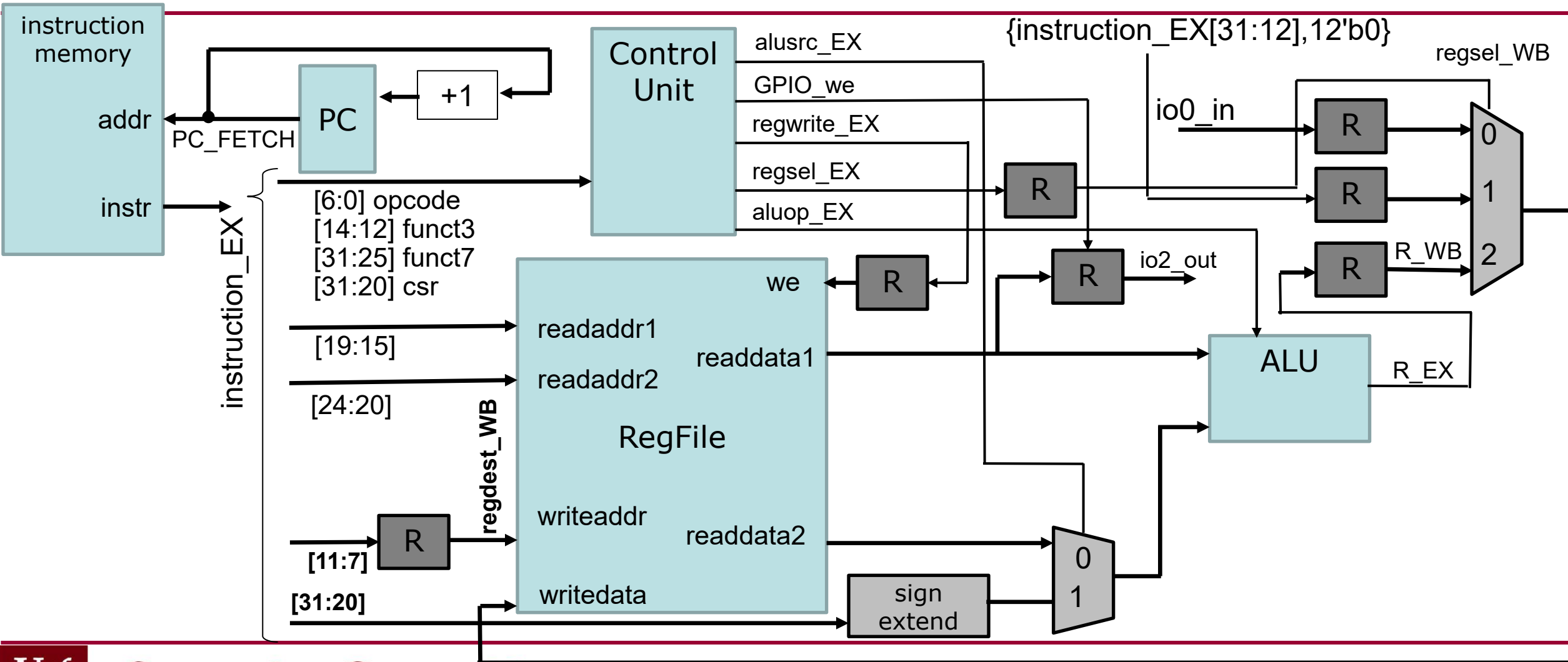
{instruction_EX[31:12], 12'b0}
or {imm20_EX, 12'b0}



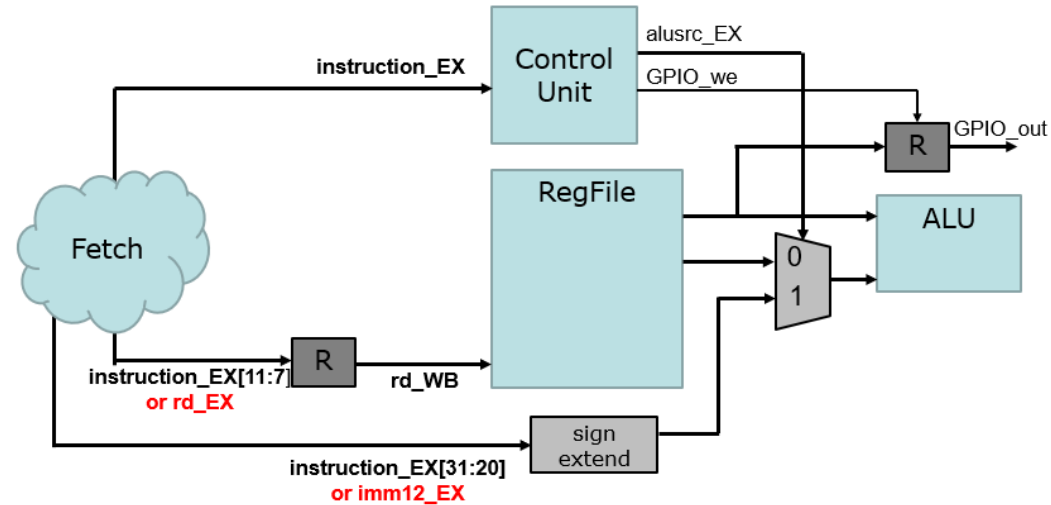
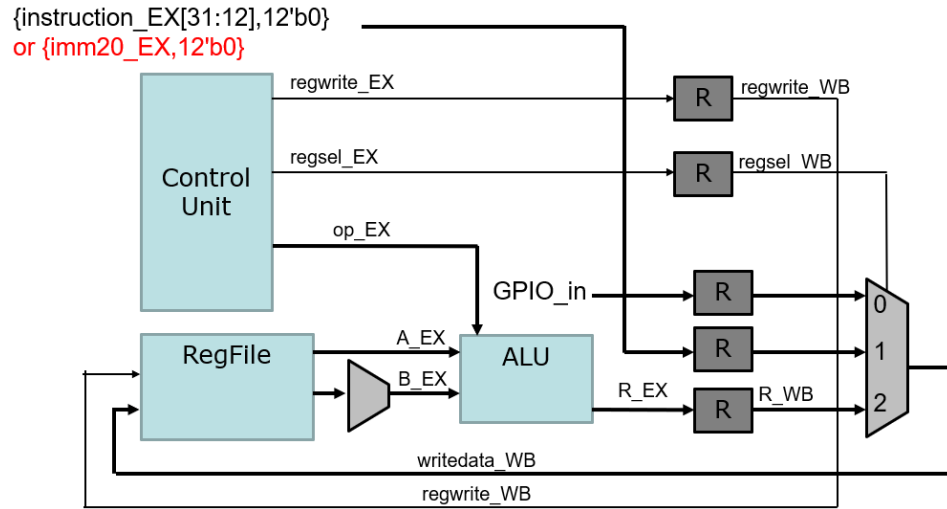
EX Stage: I-Type ARITH



Whole CPU



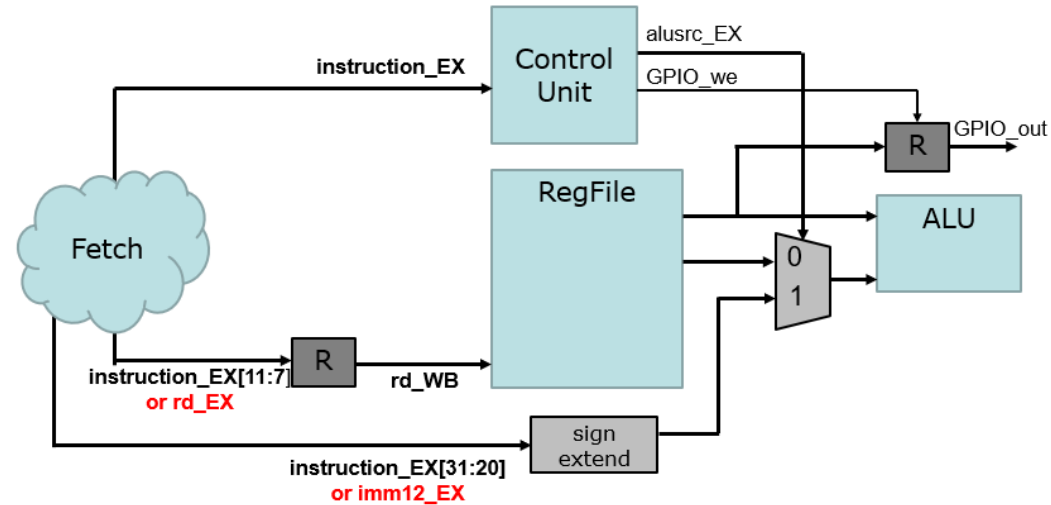
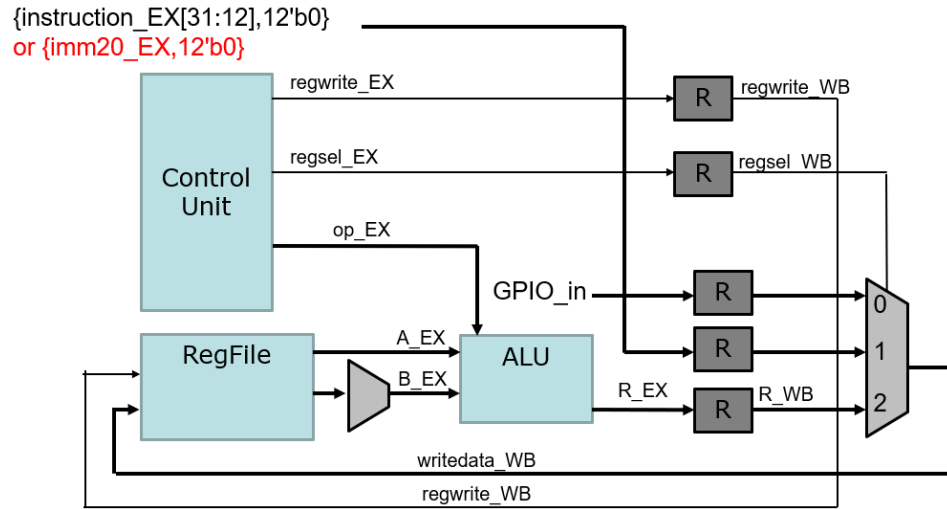
Datapaths



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33					
sub	7'h20	X	3'b000	7'h33					
mul	7'h1	X	3'b000	7'h33					
slt	7'h0	X	3'b010	7'h33					

Datapaths

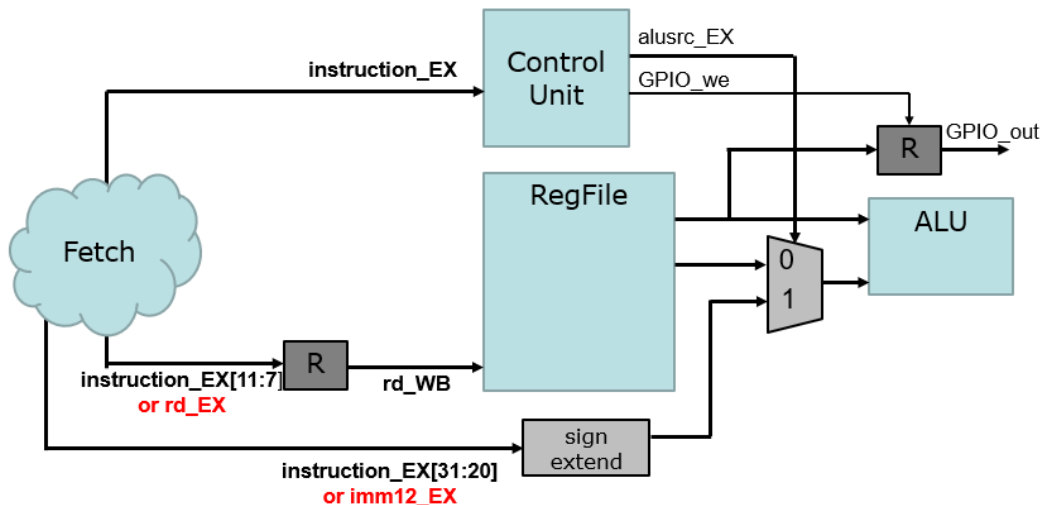
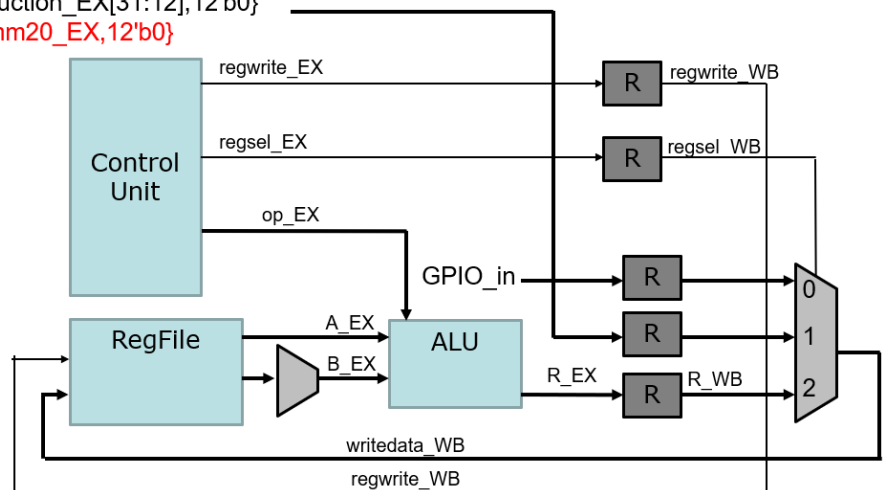


op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33	4'b0011				
sub	7'h20	X	3'b000	7'h33	4'b0100				
mul	7'h1	X	3'b000	7'h33	4'b0101				
slt	7'h0	X	3'b010	7'h33	4'b1100				

Datapaths

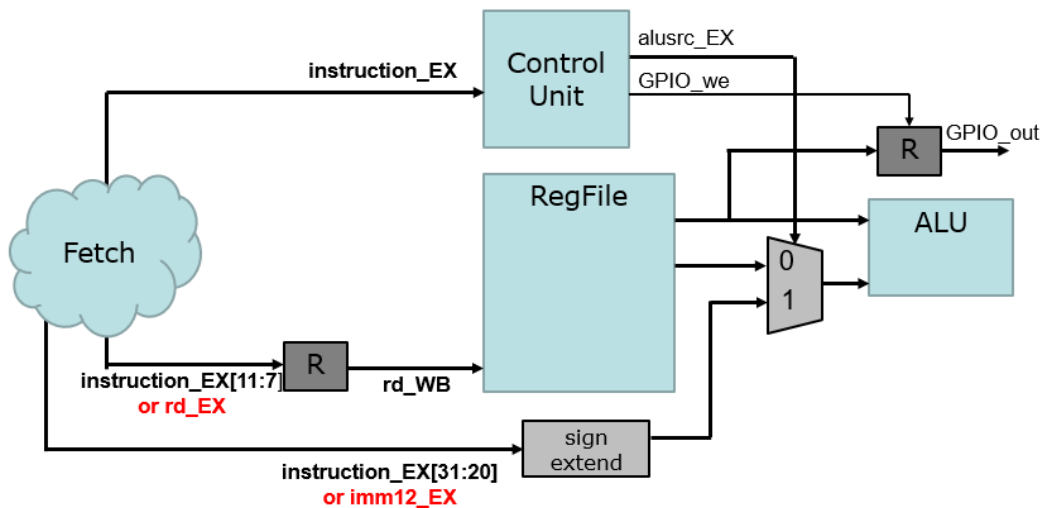
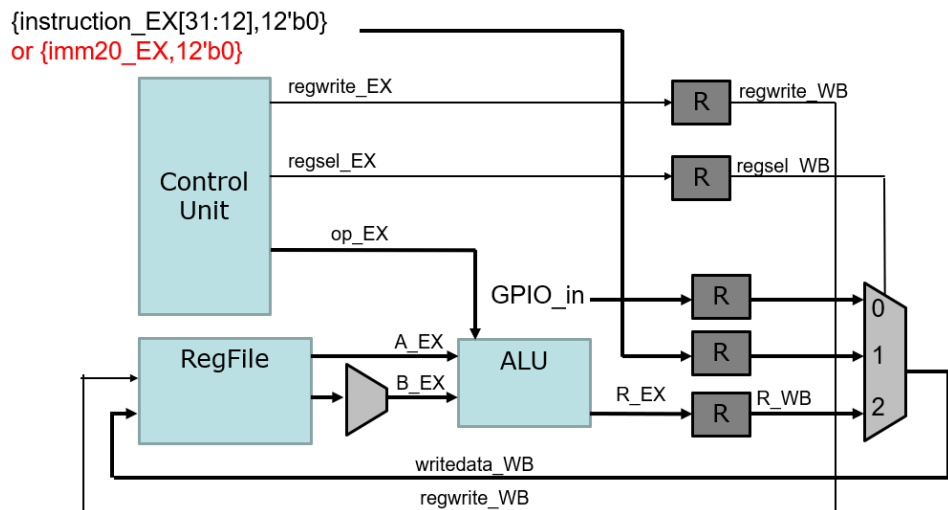
{instruction_EX[31:12], 12'b0}
or {imm20_EX, 12'b0}



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33	4'b0011	1'b0			
sub	7'h20	X	3'b000	7'h33	4'b0100	1'b0			
mul	7'h1	X	3'b000	7'h33	4'b0101	1'b0			
slt	7'h0	X	3'b010	7'h33	4'b1100	1'b0			

Datapaths

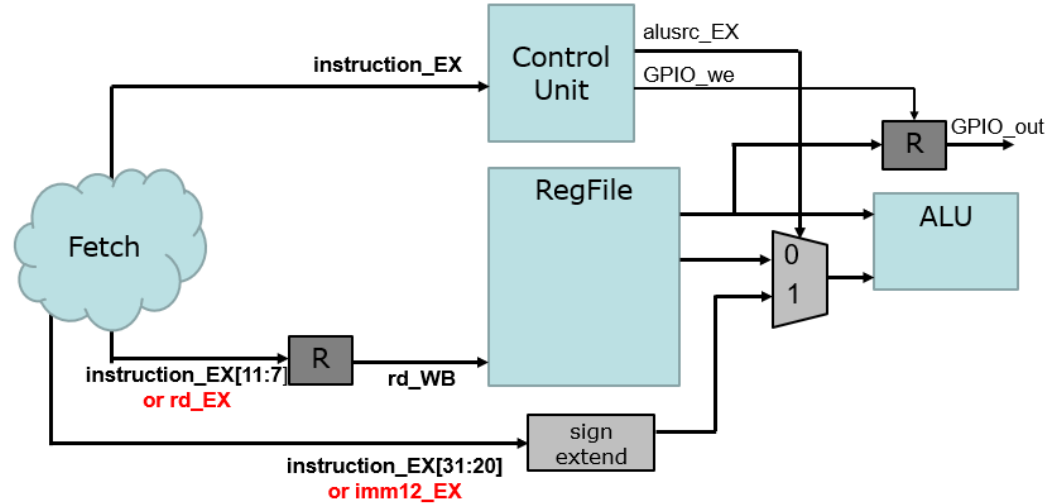
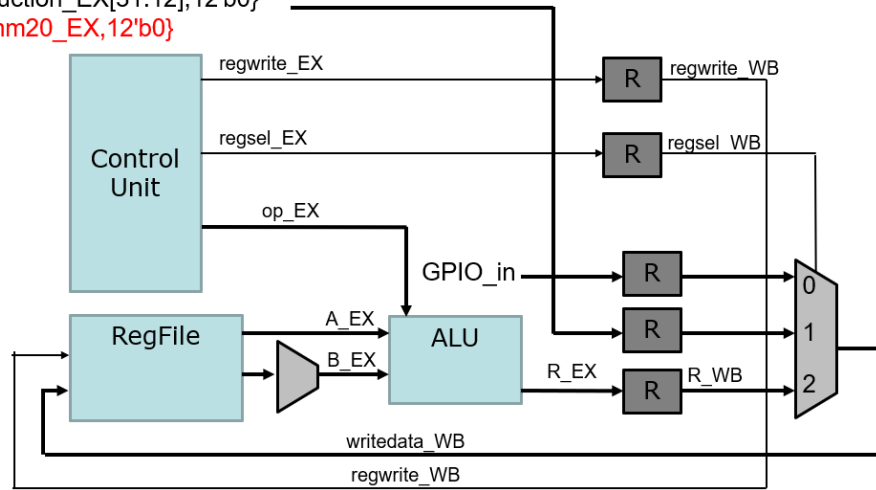


op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33	4'b0011	1'b0	2'b10		
sub	7'h20	X	3'b000	7'h33	4'b0100	1'b0	2'b10		
mul	7'h1	X	3'b000	7'h33	4'b0101	1'b0	2'b10		
slt	7'h0	X	3'b010	7'h33	4'b1100	1'b0	2'b10		

Datapaths

{instruction_EX[31:12], 12'b0}
or {imm20_EX, 12'b0}

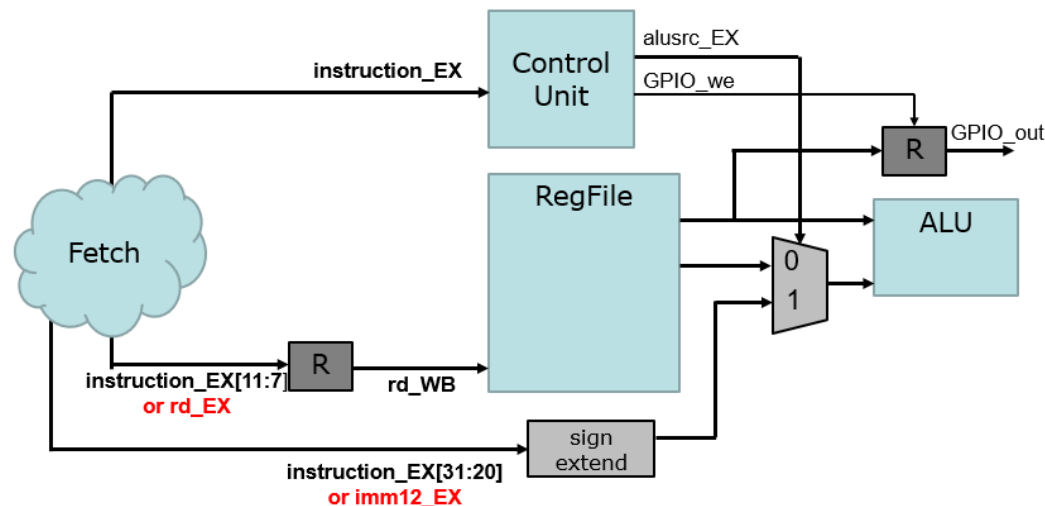
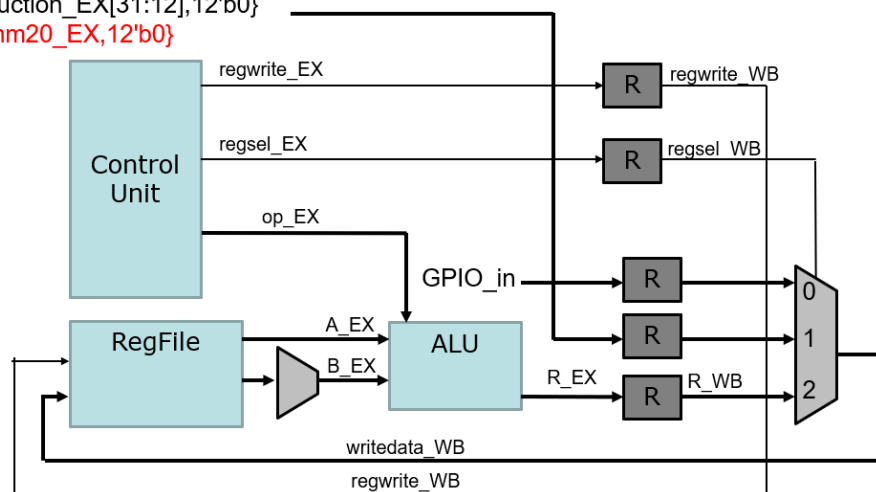


op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33	4'b0011	1'b0	2'b10	1'b1	
sub	7'h20	X	3'b000	7'h33	4'b0100	1'b0	2'b10	1'b1	
mul	7'h1	X	3'b000	7'h33	4'b0101	1'b0	2'b10	1'b1	
slt	7'h0	X	3'b010	7'h33	4'b1100	1'b0	2'b10	1'b1	

Datapaths

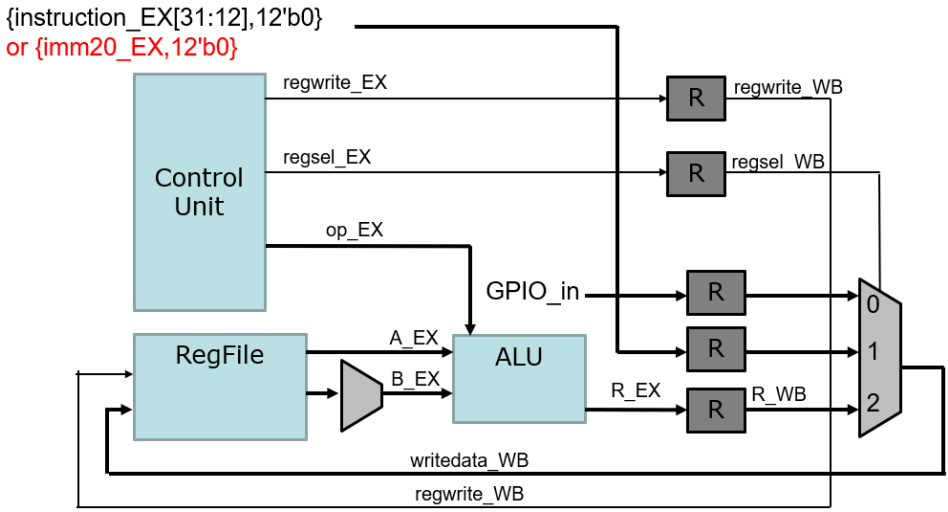
{instruction_EX[31:12], 12'b0}
or {imm20_EX, 12'b0}



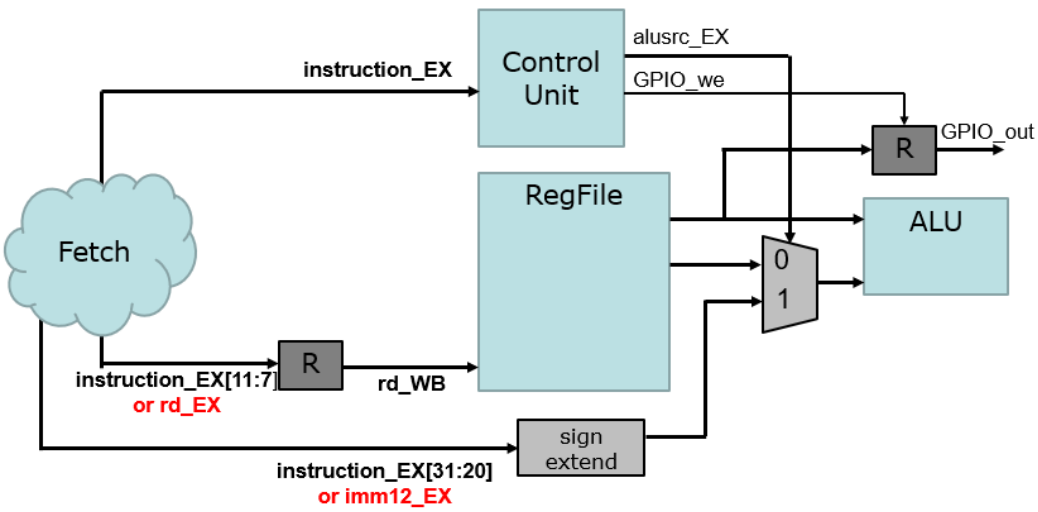
op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33	4'b0011	1'b0	2'b10	1'b1	1'b0
sub	7'h20	X	3'b000	7'h33	4'b0100	1'b0	2'b10	1'b1	1'b0
mul	7'h1	X	3'b000	7'h33	4'b0101	1'b0	2'b10	1'b1	1'b0
slt	7'h0	X	3'b010	7'h33	4'b1100	1'b0	2'b10	1'b1	1'b0

Datapaths

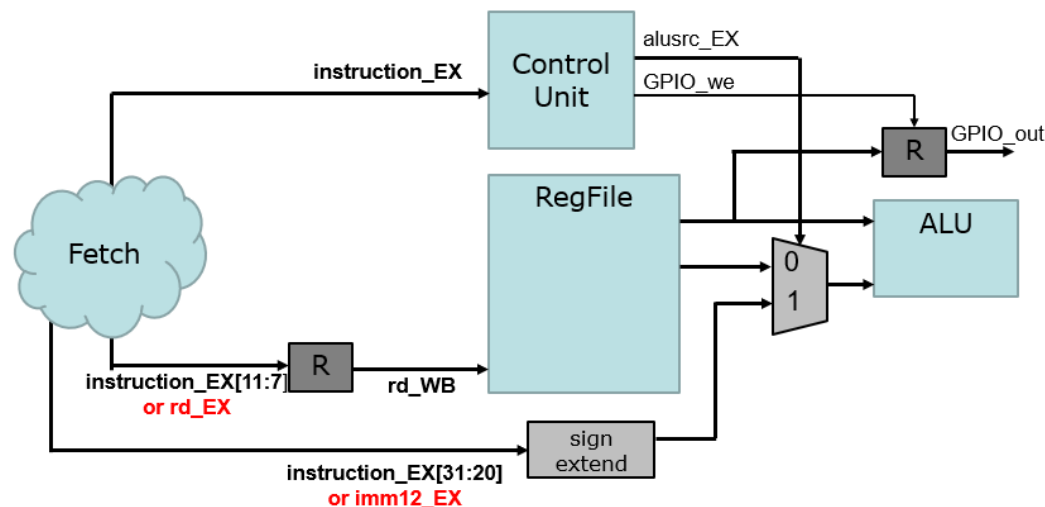
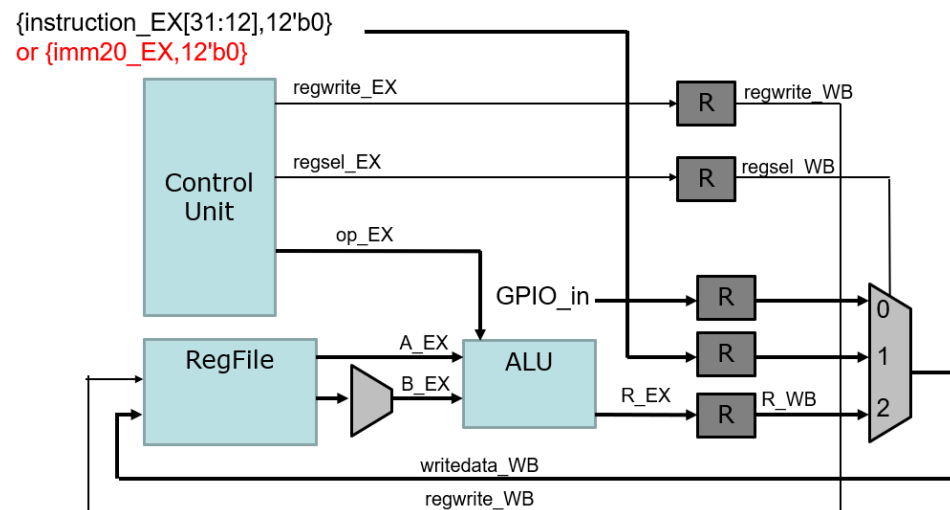


inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
and	7'h0	X	3'b111	7'h33					
andi	X	X	3'b111	7'h13					
sll	7'h0	X	3'b001	7'h33					
slli	7'h0	X	3'b001	7'h13					



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

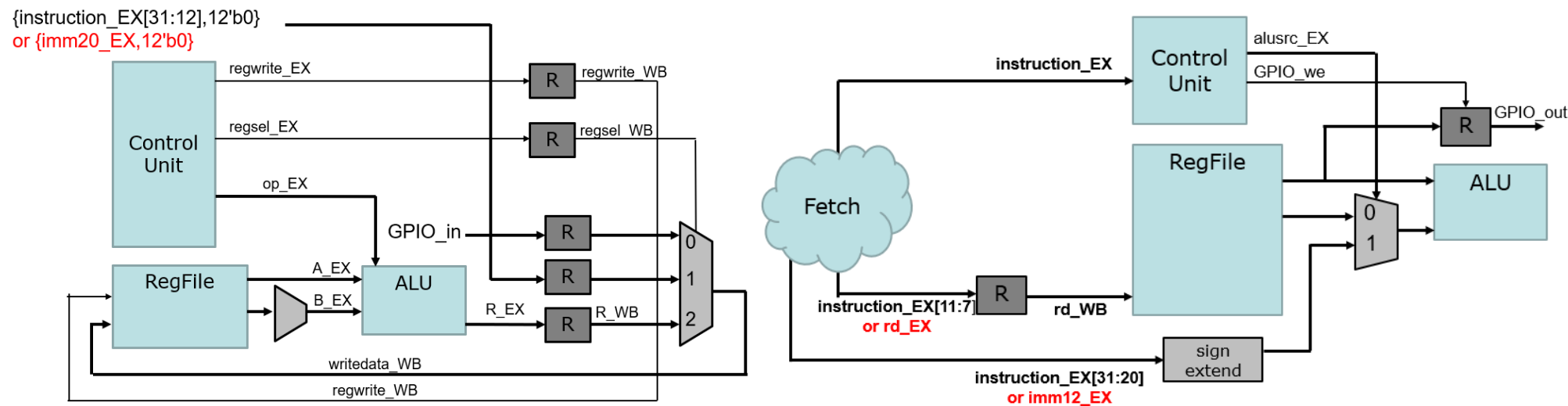
Datapaths



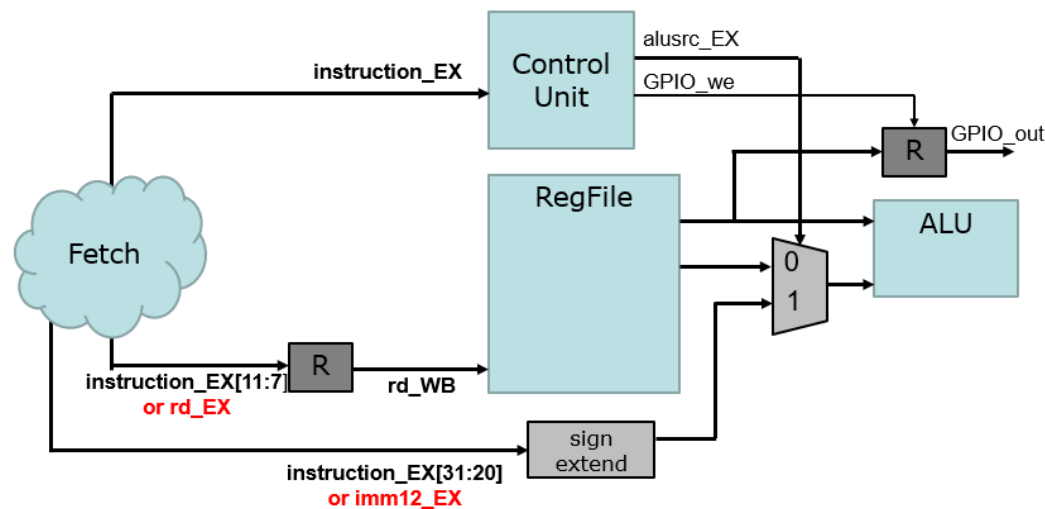
inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
and	7'h0	X	3'b111	7'h33	4'b0000				
andi	X	X	3'b111	7'h13	4'b0000				
sll	7'h0	X	3'b001	7'h33	4'b1000				
slli	7'h0	X	3'b001	7'h13	4'b1000				

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

Datapaths

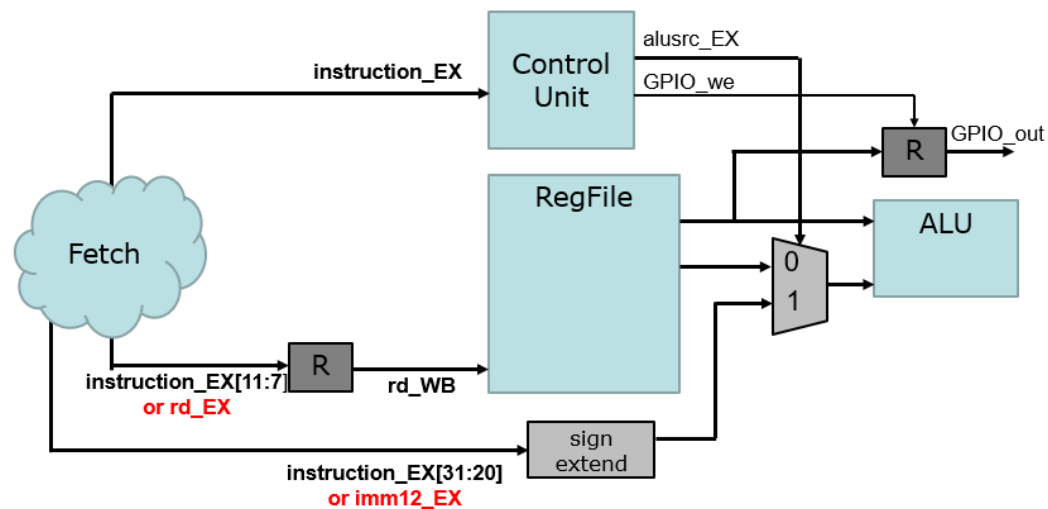
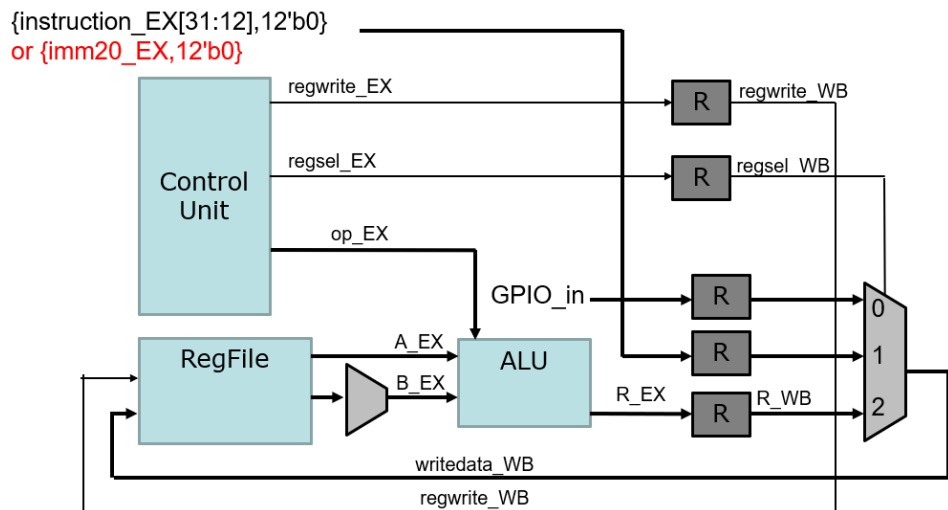


inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
and	7'h0	X	3'b111	7'h33	4'b0000	1'b0			
andi	X	X	3'b111	7'h13	4'b0000	1'b1			
sll	7'h0	X	3'b001	7'h33	4'b1000	1'b0			
slli	7'h0	X	3'b001	7'h13	4'b1000	1'b1			



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

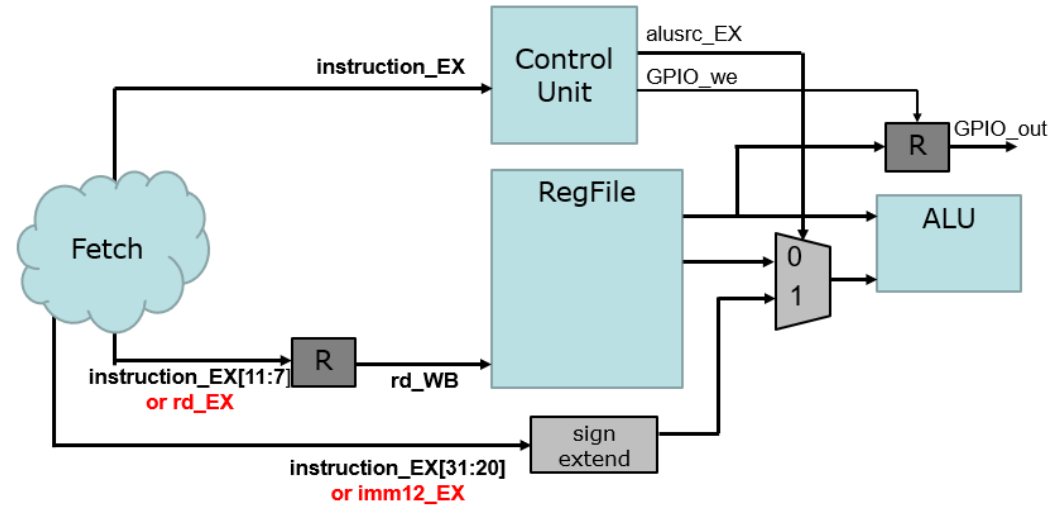
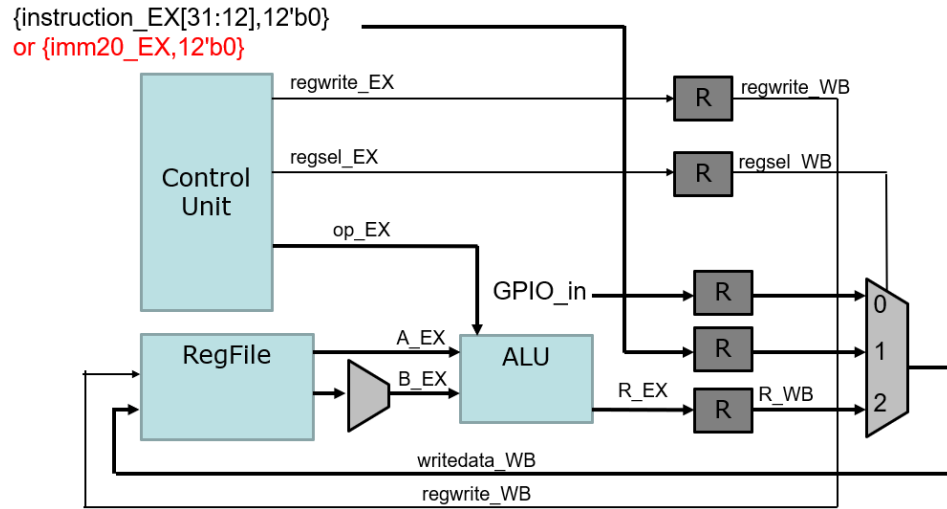
Datapaths



inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
and	7'h0	X	3'b111	7'h33	4'b0000	1'b0	2'b10		
andi	X	X	3'b111	7'h13	4'b0000	1'b1	2'b10		
sll	7'h0	X	3'b001	7'h33	4'b1000	1'b0	2'b10		
slli	7'h0	X	3'b001	7'h13	4'b1000	1'b1	2'b10		

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

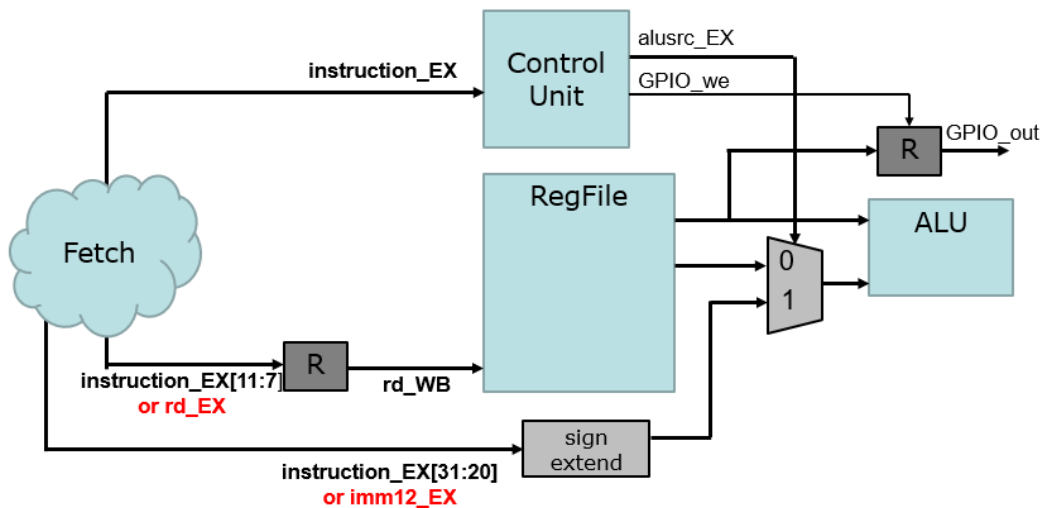
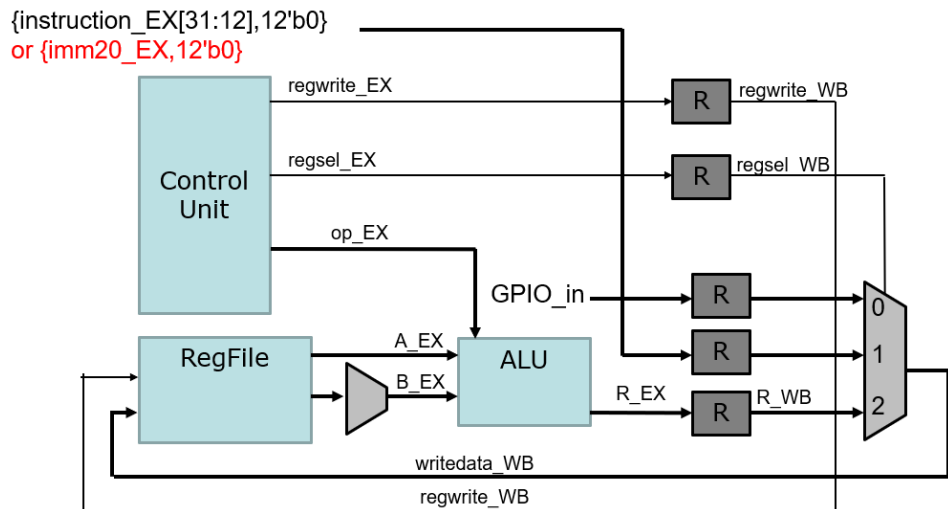
Datapaths



inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
and	7'h0	X	3'b111	7'h33	4'b0000	1'b0	2'b10	1'b1	
andi	X	X	3'b111	7'h13	4'b0000	1'b1	2'b10	1'b1	
sll	7'h0	X	3'b001	7'h33	4'b1000	1'b0	2'b10	1'b1	
slli	7'h0	X	3'b001	7'h13	4'b1000	1'b1	2'b10	1'b1	

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

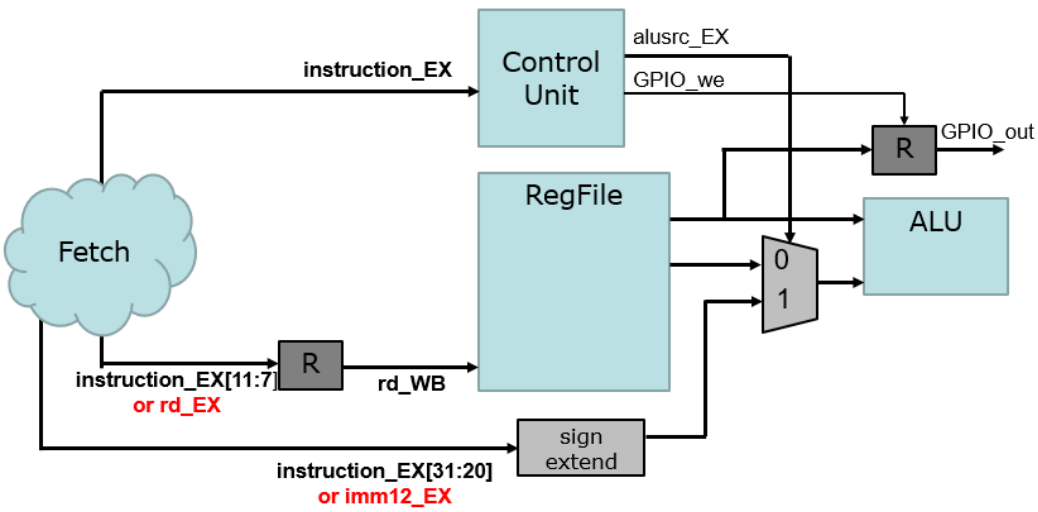
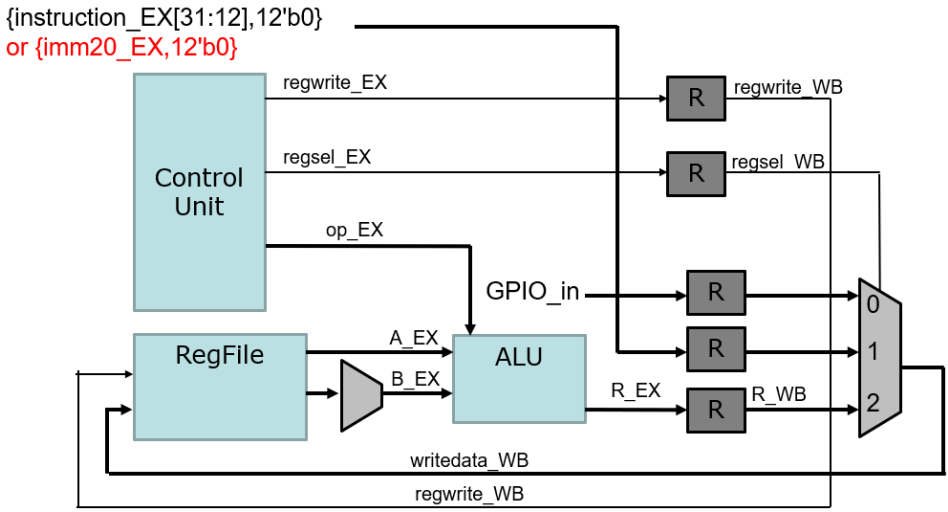
Datapaths



inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
and	7'h0	X	3'b111	7'h33	4'b0000	1'b0	2'b10	1'b1	1'b0
andi	X	X	3'b111	7'h13	4'b0000	1'b1	2'b10	1'b1	1'b0
sll	7'h0	X	3'b001	7'h33	4'b1000	1'b0	2'b10	1'b1	1'b0
slli	7'h0	X	3'b001	7'h13	4'b1000	1'b1	2'b10	1'b1	1'b0

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

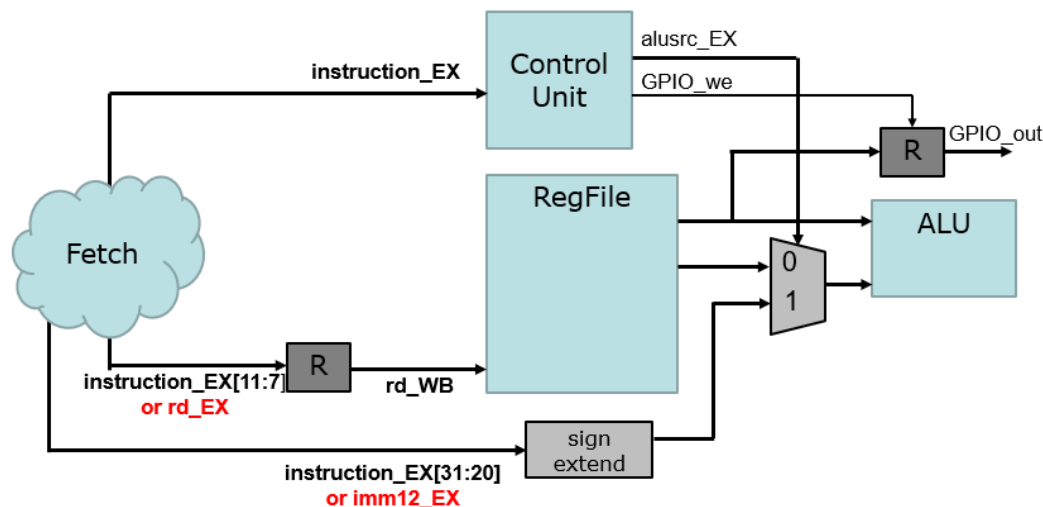
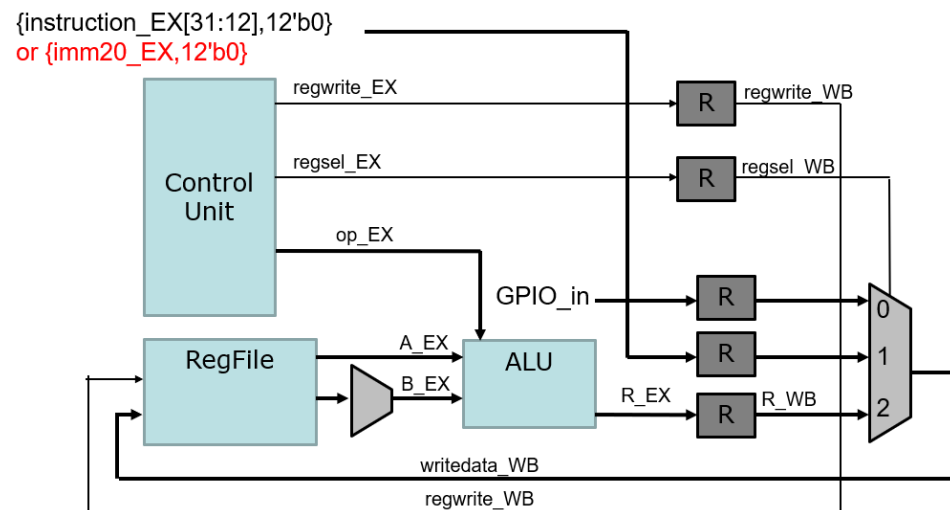
Datapaths



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
lui	X	X	X	7'h37					
csrrw HEX	X	12'hf02 (io2)	3'b001	7'h73					
csrrw SW	X	12'hf00 (io0)	3'b001	7'h73					

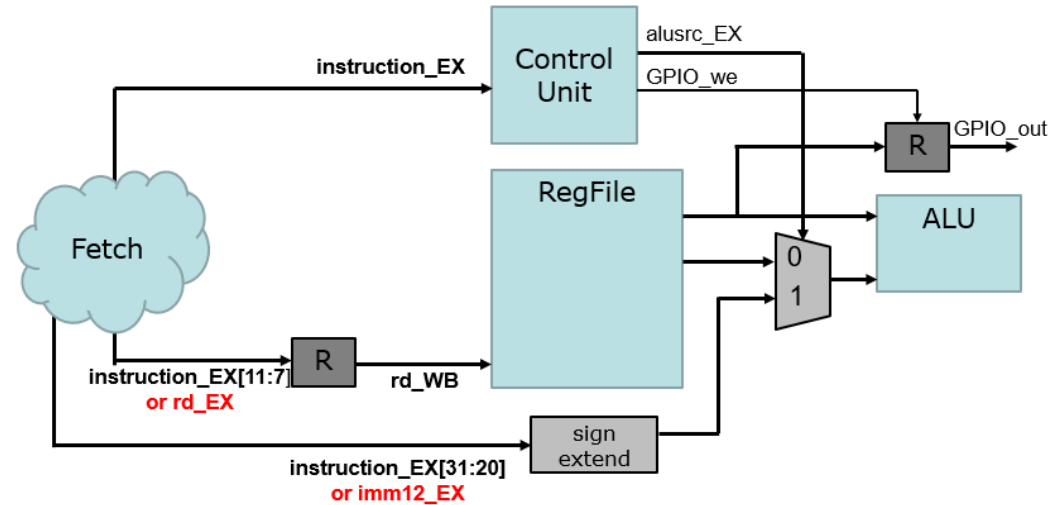
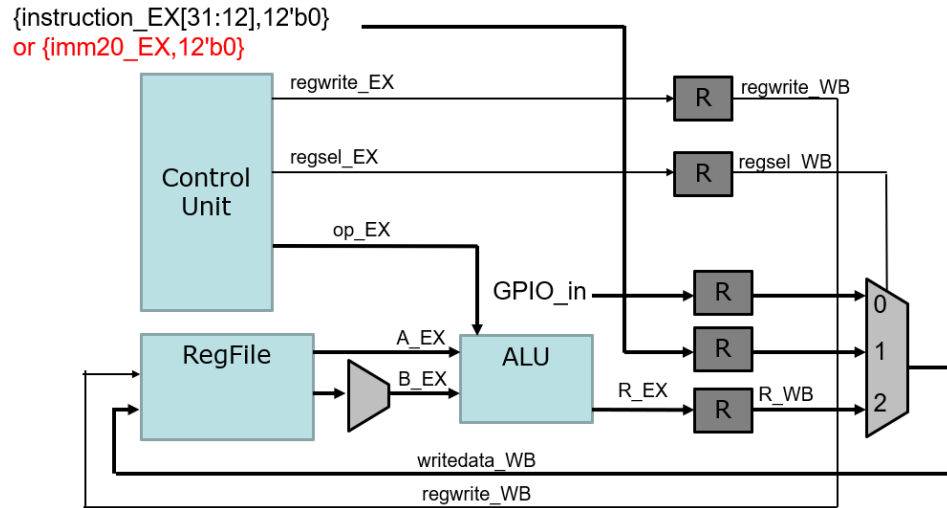
Datapaths



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
lui	X	X	X	7'h37	4'bX				
csrrw HEX	X	12'hf02 (io2)	3'b001	7'h73	4'bX				
csrrw SW	X	12'hf00 (io0)	3'b001	7'h73	4'bX				

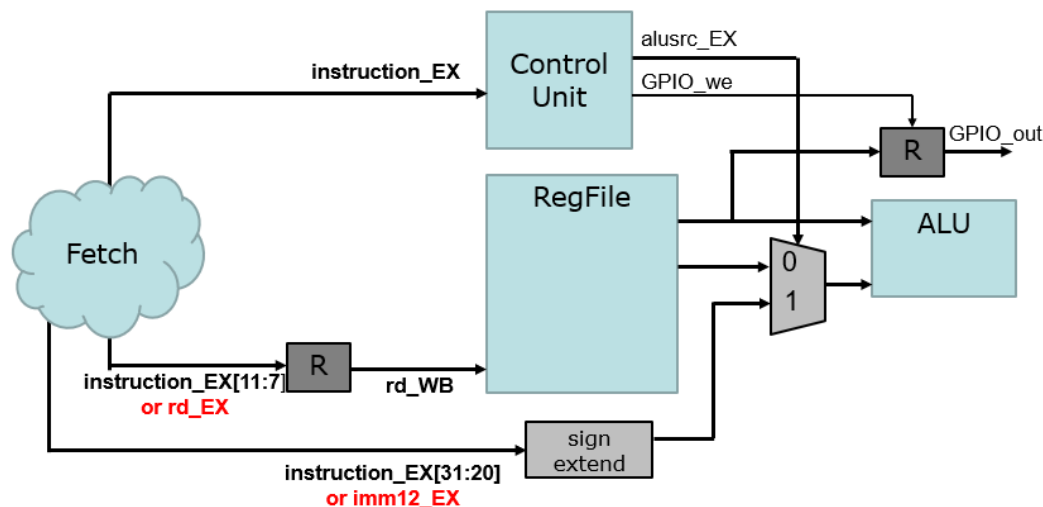
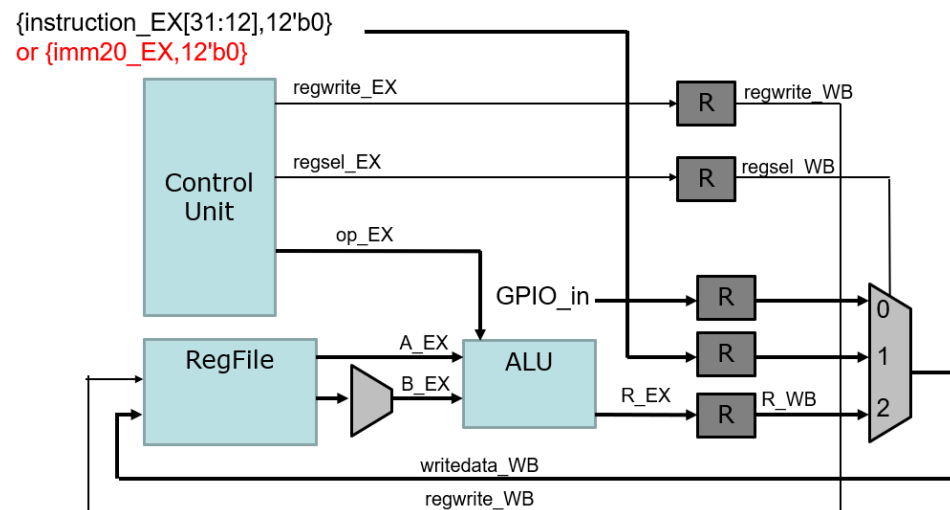
Datapaths



inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
lui	X	X	X	7'h37	4'bX	1'bX			
crrw HEX	X	12'hf02 (io2)	3'b001	7'h73	4'bX	1'bX			
crrw SW	X	12'hf00 (io0)	3'b001	7'h73	4'bX	1'bX			

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

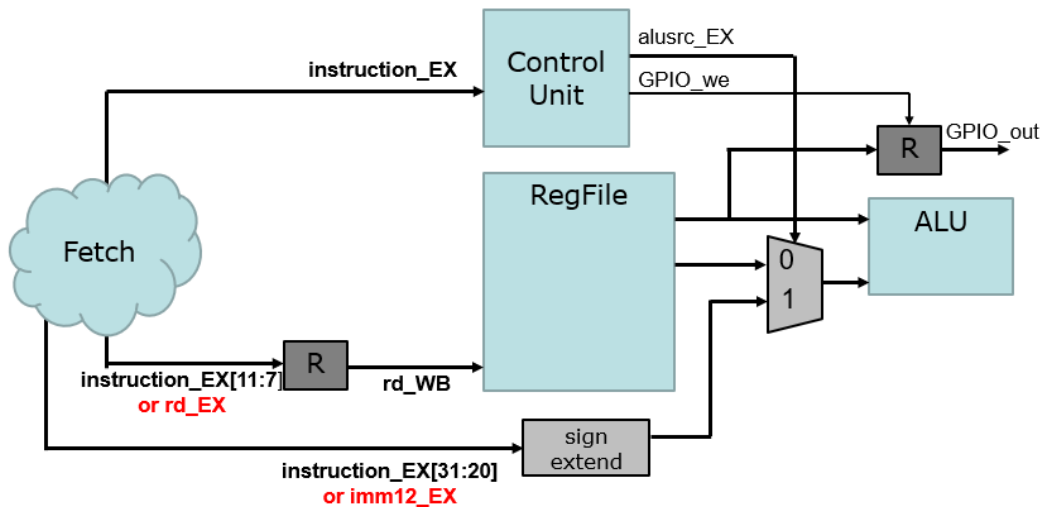
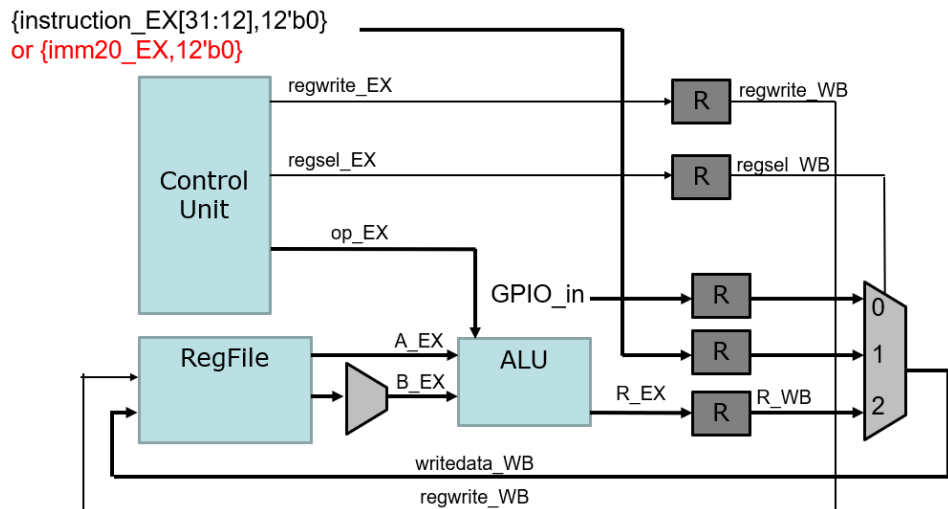
Datapaths



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
lui	X	X	X	7'h37	4'bX	1'bX	2'b01		
csrrw HEX	X	12'hf02 (io2)	3'b001	7'h73	4'bX	1'bX	2'bX		
csrrw SW	X	12'hf00 (io0)	3'b001	7'h73	4'bX	1'bX	2'b00		

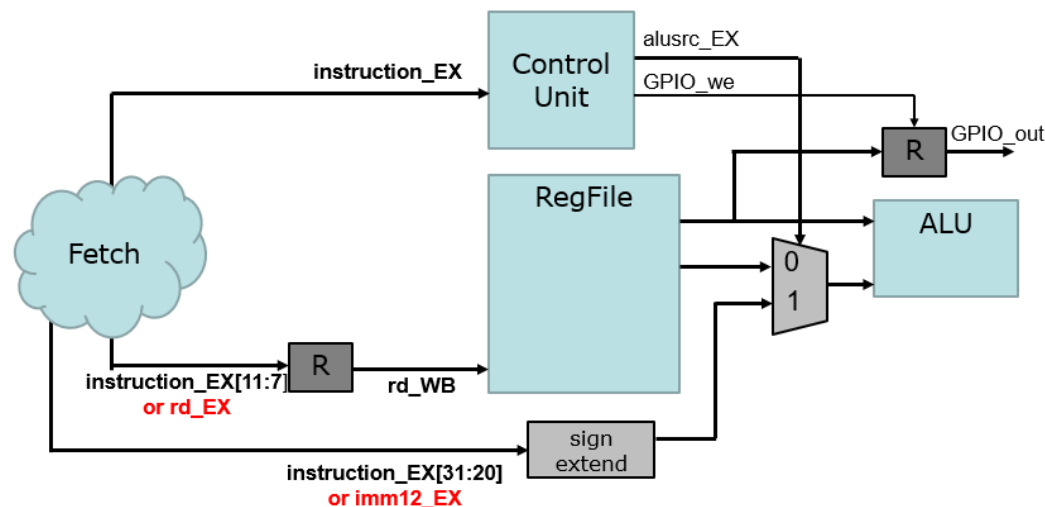
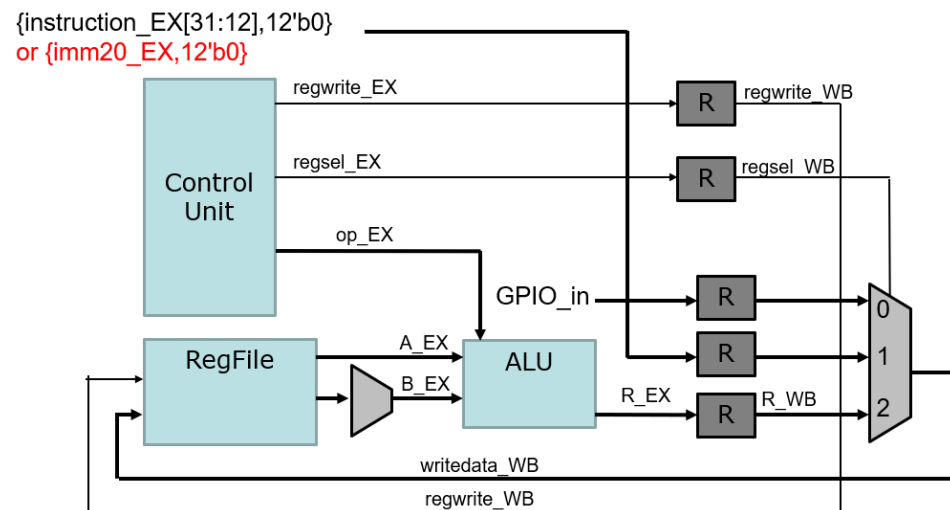
Datapaths



inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
lui	X	X	X	7'h37	4'bX	1'bX	2'b01	1'b1	
csrrw HEX	X	12'hf02 (io2)	3'b001	7'h73	4'bX	1'bX	2'bX	1'b0	
csrrw SW	X	12'hf00 (io0)	3'b001	7'h73	4'bX	1'bX	2'b00	1'b1	

op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A – B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

Datapaths



op	Function
0000	A and B
0001	A or B
0010	A xor B
0011	A + B
0100	A - B
0101	A * B (low)
0110	A * B (high, signed)
0111	A * B (high, unsigned)
1000	A << B
1001	A >> B
1010	A >>> B
1011	A >>> B
1100	A < B (signed)
1101	A < B (unsigned)
1110	A < B (unsigned)
1111	A < B (unsigned)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
lui	X	X	X	7'h37	4'bX	1'bX	2'b01	1'b1	1'b0
csrrw HEX	X	12'hf02 (io2)	3'b001	7'h73	4'bX	1'bX	2'bX	1'b0	1'b1
csrrw SW	X	12'hf00 (io0)	3'b001	7'h73	4'bX	1'bX	2'b00	1'b1	1'b0

Control Unit Implementation (NOT COMPLETE)

inst	inst 31:25 funct7	inst 31:20 (imm12)	inst 14:12 funct3	inst 6:0 opcode	aluop	alusrc	regsel	regwrite	gpio_we
add	7'h0	X	3'b000	7'h33	4'b0011	1'b0	2'b10	1'b1	1'b0
sub	7'h20	X	3'b000	7'h33	4'b0100	1'b0	2'b10	1'b1	1'b0
mul	7'h1	X	3'b000	7'h33	4'b0101	1'b0	2'b10	1'b1	1'b0
slt	7'h0	X	3'b010	7'h33	4'b1100	1'b0	2'b10	1'b1	1'b0
and	7'h0	X	3'b111	7'h33	4'b0000	1'b0	2'b10	1'b1	1'b0
andi	X	X	3'b111	7'h13	4'b0000	1'b1	2'b10	1'b1	1'b0
sll	7'h0	X	3'b001	7'h33	4'b1000	1'b0	2'b10	1'b1	1'b0
slli	7'h0	X	3'b001	7'h13	4'b1000	1'b1	2'b10	1'b1	1'b0
lui	X	X	X	7'h37	4'bX	1'bX	2'b01	1'b1	1'b0
csrrw HEX	X	12'hf02 (io2)	3'b001	7'h73	4'bX	1'bX	4'bX	1'b0	1'b1
csrrw SW	X	12'hf00 (io0)	3'b001	7'h73	4'bX	1'bX	2'b00	1'b1	1'b0

Example Test Program

```
.text
addi x1, x0, 12
addi x2, x0, 17
add x3, x1, x2
sub x3, x3, x1
slli x3, x3, 27
mul x4, x2, x3
mulh x4, x2, x3
mulhu x5, x2, x3
slt x6, x4, x1
sltu x6, x4, x1
and x7, x4, x1
or x8, x4, x1
xor x9, x1, x2
andi x10, x3, -2048
ori x11, x2, -2048
xori x12, x2, 14
sll x13, x2, x1
srl x14, x3, x2
sra x15, x3, x2
slli x16, x1, 1
srli x17, x2, 1
srai x18, x3, 1
lui x19, 0xDBEEF
csrrw x20, 0xf02, x3
csrrw x21, 0xf00, x3

# (x1) <= 12 / 0x0000000c
# (x2) <= 17 / 0x00000011
# (x3) <= 29 / 0x0000001D
# (x3) <= 17 / 0x00000011
# (x3) <= -2013265920 / 0x88000000
# (x4) <= 134217728 / 0x08000000
# (x4) <= -8 / 0xFFFFFFFF8
# (x5) <= 9 / 0x00000009
# (x6) <= 1 / 0x00000001
# (x6) <= 0 / 0x00000000
# (x7) <= 8 / 0x00000008
# (x8) <= -4 / 0xFFFFFFFFC
# (x9) <= 29 / 0x0000001D
# (x10) <= -2013265920 / 0x88000000
# (x11) <= -2031 / 0xFFFFF811
# (x12) <= 31 / 0x0000001F
# (x13) <= 69632 / 0x00011000
# (x14) <= 17408 / 0x00004400
# (x15) <= -15360 / 0xFFFFC400
# (x16) <= 24 / 0x00000018
# (x17) <= 8 / 0x00000008
# (x18) <= -1006632960 / 0xC4000000
# (x19) <= -605097984 / 0xDBEEF000
# HEX <= 0x88000000 / -2013265920
# (x21) <= SW
```

Binary to Decimal Conversion

val	%10	/10
5678	8	567
567	7	56
56	6	5
5	5	0
0		

- `reg=0`
- `while val != 0`
 - `reg = reg << 4`
 - `reg = reg | val % 10`
 - `val = val / 10`
- This will generate the digits in reverse, so you can connect the least significant BCD digit of the register to the most significant HEX

Work Around Divide

- Our CPU doesn't have a divide or a modulo instruction
- Since we're using a constant factor 10, we can use multiply

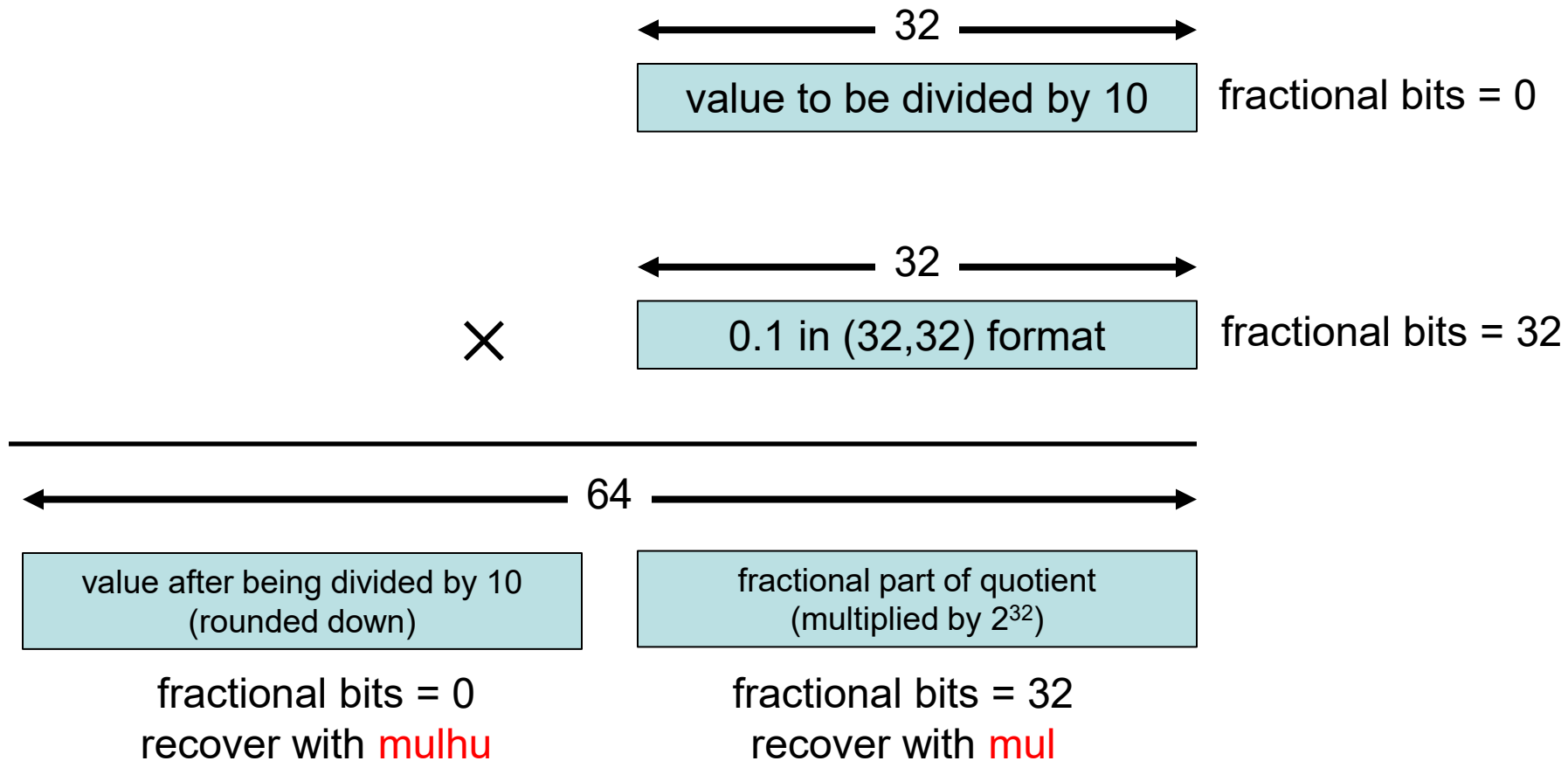
- Idea: use fixed-point multiply
 - Assume a decimal point at some location in a value:
 - Example (6,4)-fixed format:

1	0.	1	1	0	1
---	----	---	---	---	---

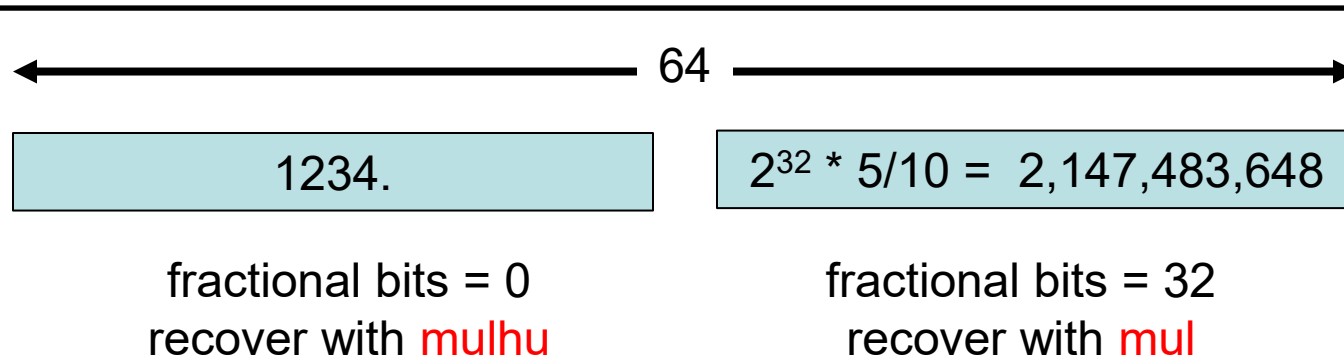
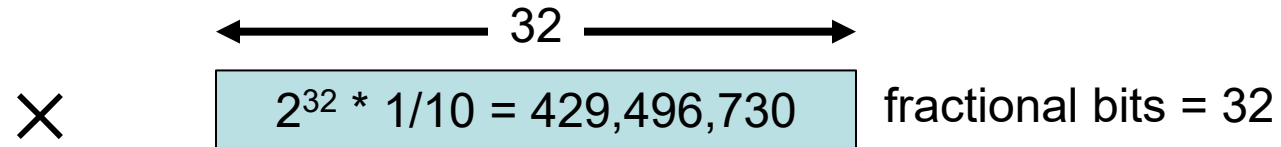
- $= 2 + 13/16 = 2.8125$

- Now assume we multiply a (32,0) value by a (32,32) value
 - Result would be (64,32) value
 - Use this to multiply a (32,0) value by 0.1
 - (32,32) representation of 0.1 = $2^{32} / 10$

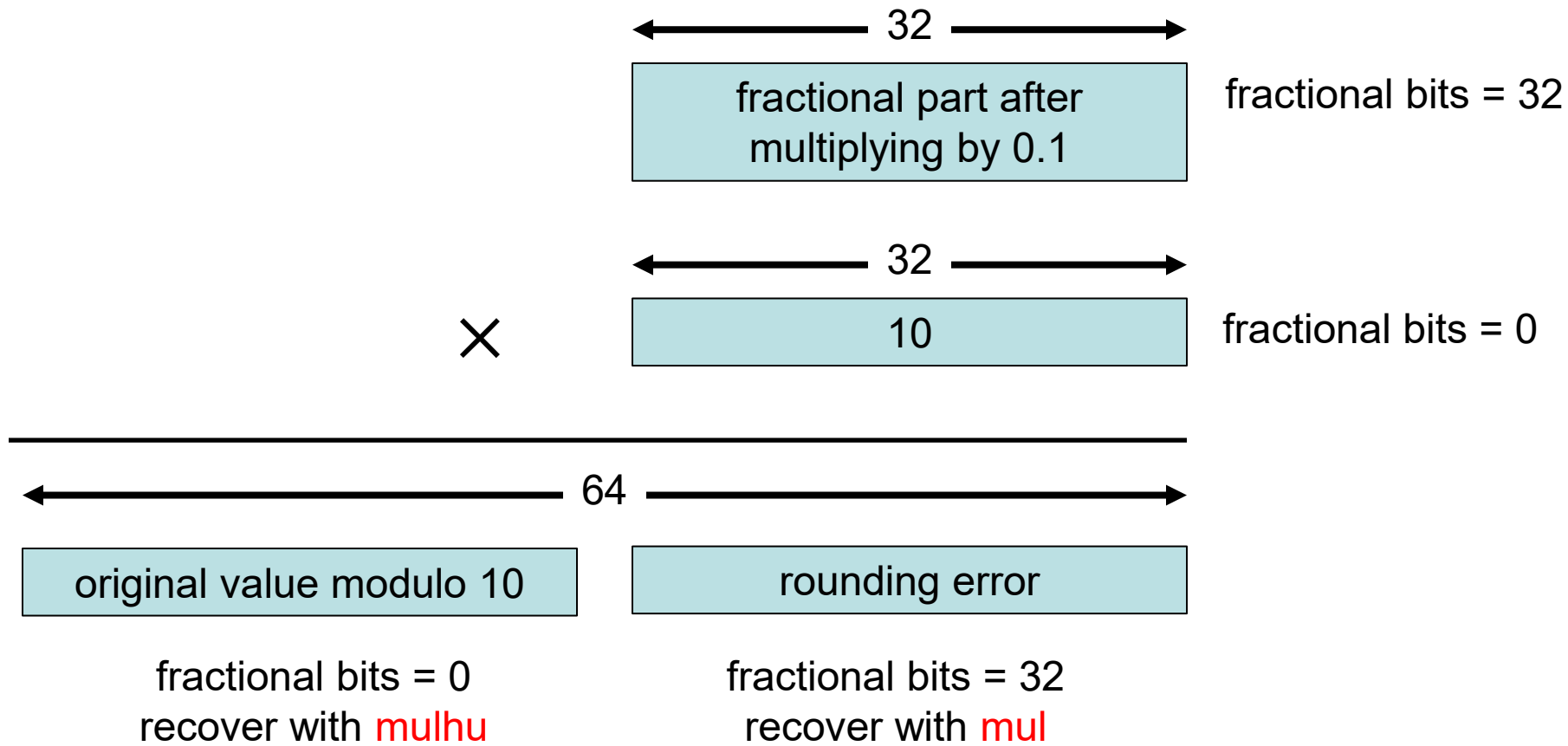
Binary to Base-10



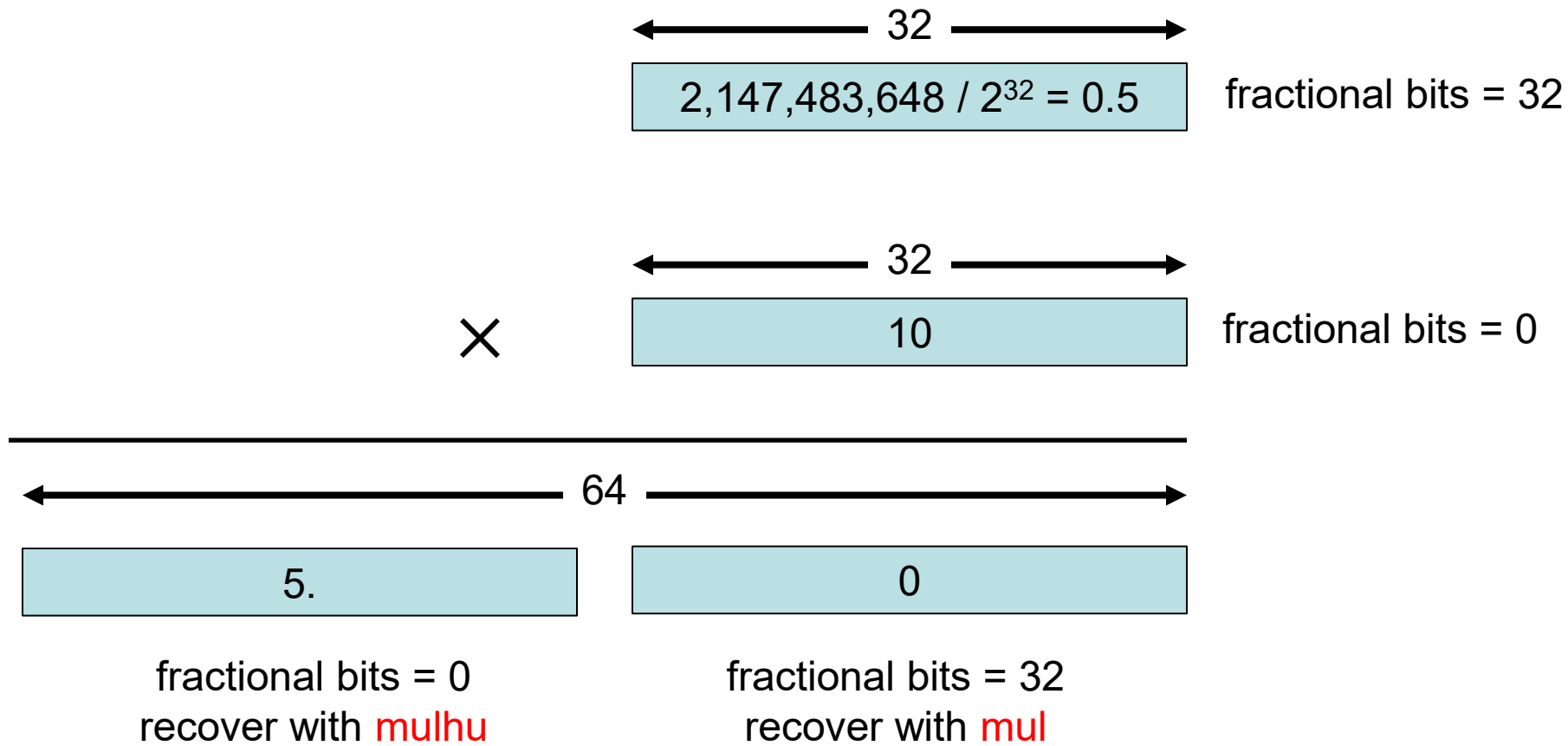
Binary to Base-10



Binary to Base-10



Binary to Base-10



Example

- Assume value = 234
 - Step 1: $\text{temp} = 234 \times 0.1 = 23.4$
 - Step 2: $\text{temp2} = \text{fractional}(\text{temp}) = .4$
 - Step 3: $\text{temp} = \text{whole}(\text{temp}) = 23$
 - Step 4: $\text{digit} = \text{temp2} \times 10 = 4$
 - Step 5: $\text{temp} = \text{temp} \times 0.1 = 2.3$
 - Step 6: $\text{temp2} = \text{fractional}(\text{temp}) = .3$
 - Step 7: $\text{temp} = \text{whole}(\text{temp}) = 2$
 - Step 8: $\text{digit} = \text{temp2} \times 10 = 3$
 - Step 9: $\text{temp} = \text{temp} \times 0.1 = 0.2$
 - Step 10: $\text{temp2} = \text{fractional}(\text{temp}) = .2$
 - Step 11: $\text{temp} = \text{whole}(\text{temp}) = 0$
 - Step 12: $\text{digit} = \text{temp2} \times 10 = 2$
 - Step 13: $\text{temp} == 0$ so finish

Tips for Lab 3

- To record all signals during simulation, add the following line in your tesbench's initial block:

```
$wlfdumpvars(0,tb);
```

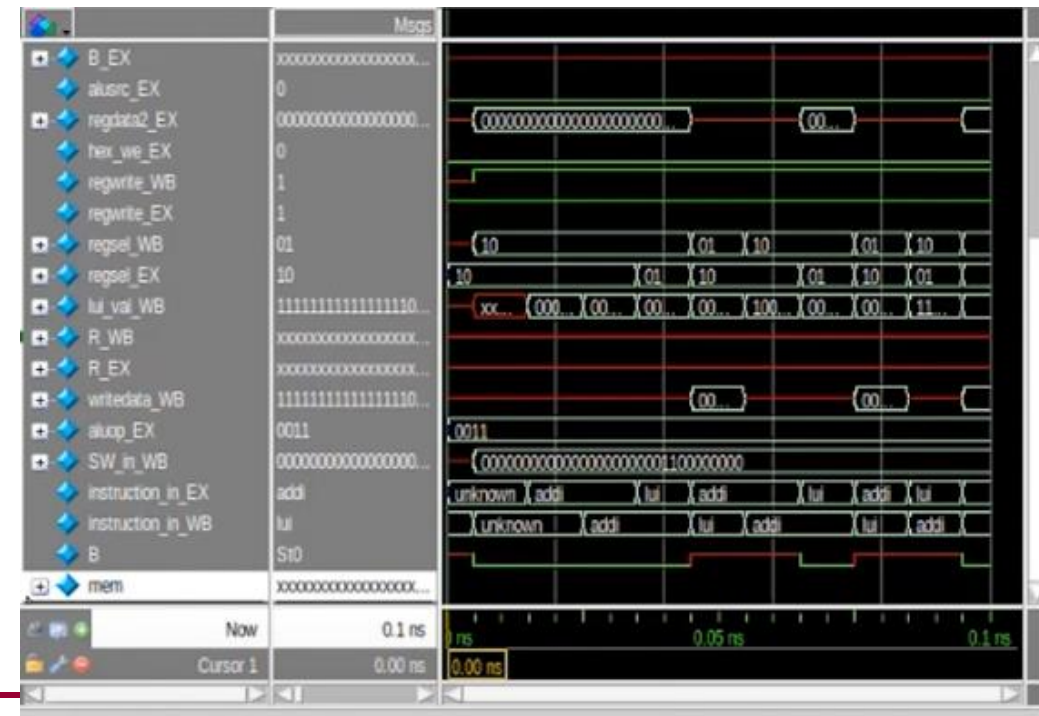
- To aid in debugging:

```
typedef enum  
{add,sub,slt,and_,andi,sll,slli,lui,csrrw_hex,csrrw_sw,addi,  
srli} instruction_name;
```

```
instruction_name instruction_in_EX, instruction_in_WB;  
always @(posedge clk)  
    instruction_in_WB <= instruction_in_EX;
```

- In control unit, for example:

```
instruction_in_EX = sub;
```



Branch Instructions

- Branch offset = immediate field
 - Number of instructions to advance the PC if the branch is taken
 - Which PC?
 - PC_FETCH: the address of the instruction currently being fetched
 - PC_EX: the address of the instruction currently being executed (in the execute stage)

RISC-V:

Text Segment				
Bkpt	Address	Code	Basic	
☐	0x00400000	0x00730063	beq x6,x7,0x00000000	l: foo: beq t1,t2,foo

MIPS:

Text Segment				
Bkpt	Address	Code	Basic	
☐	0x00400000	0x112affff	beq \$9,\$10,0xffffffff	l: foo: beq \$t1,\$t2,foo

- RISC-V: use PC_EX
- MIPS: use PC_FETCH

Branch Instructions

- Branch instructions:
 - B-type



- beq, bne, blt, bge, bltu, bgeu
- Ex: **beq x2, x3, loop**
- If taken:

```
PC_FETCH <= PC_EX + SE(imm<<1)
    // assuming PC has byte address ^^^
instruction_EX <= NOP
```

Branch Instructions

imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode
--------------	-----	-----	--------	-------------	--------

instruction_EX bits	offset bits
31	12
30:25	10:5
11:8	4:1
7	11

instruction_EX bits	offset bits
31	12
7	11
30:25	10:5
11:8	4:1

- 12-bit immediate (rep. 13-bit offset):

```
assign branch_offset_EX =
```

```
{instruction_EX[31], instruction_EX[7], instruction_EX[30:25], instruction_EX[11:8], 1'b0};
```

```
assign branch_addr_EX = PC_EX + {branch_offset_EX[12], branch_offset_EX[12:2]};
```

Jump Instruction

- Jump-and-link (jal)

- J-type:



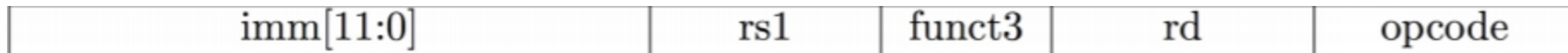
- 20-bit immediate (rep. 21-bit offset)
 - $R[rd] \leq PC_FETCH$
 - $PC_FETCH \leq PC_EX + SE(imm \ll 1)$ // assuming byte-address
 - `assign jal_offset_EX = {instruction_EX[31],instruction_EX[19:12],instruction_EX[20],instruction_EX[30:21],1'b0};`
 - `assign jal_addr_EX = PC_EX + jal_offset_EX[13:2];`

instruction_EX bits	offset bits
31	20
30:21	10:1
20	11
19:12	19:12

instruction_EX bits	offset bits
31	20
19:12	19:12
20	11
30:21	10:1

Jump Register Instruction

- Jump-and-link-register (jalr)
- I-type:



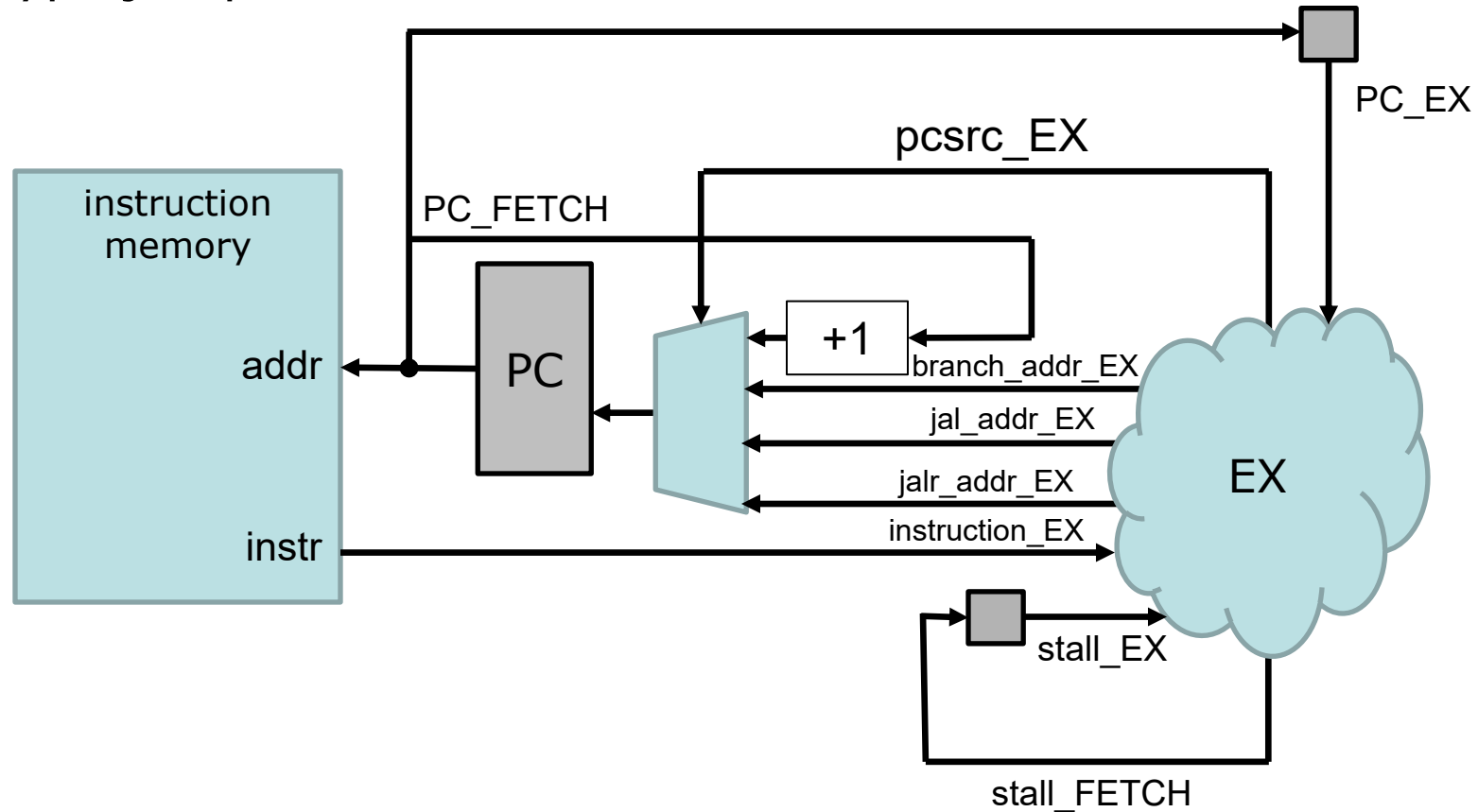
- $PC_FETCH \leq [R[rs1] + SE(imm)] \& 0xffff\ fffe$ // assuming byte address
- $R[rd] \leq PC_FETCH$
- `assign jalr_offset_EX = instruction_EX[31:20];`
- `assign jalr_addr_EX = readdatal_EX[11:0] + {{2{jlr_offset[11]}},jalr_offset[11:2]};`

Three Stage Pipeline

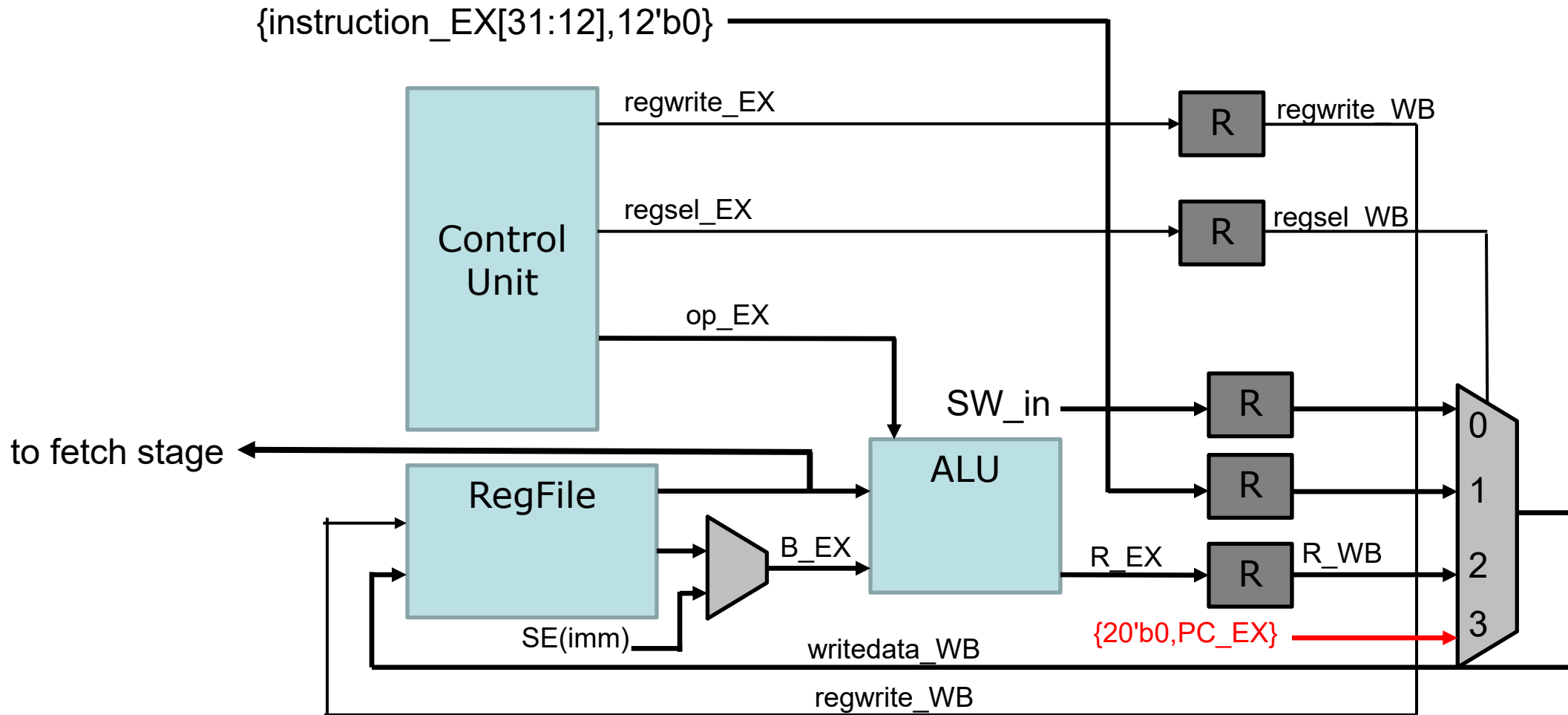
cycle	0	1	2	3	4	5	6	7
add	F	E	W					
add		F	E	W				
branch (t)			F	E	W			
fall through				F	E(S)	W(S)		
target					F	E	W	
branch(nt)						F	E	W

Fetch Stage

- Add a separate bus for branch vs. j-type jumps vs. r-type jumps



EX/WB Stage



Changes to CPU

- Structure:
 - Add branch_addr, jal_addr, and jalr_addr as possible next state values for PC_FETCH
 - Add pcsrc_EX to control PC mux
 - Add stall_FETCH as output of control unit
 - Add stall_EX as input to control unit
 - Add PC_EX as 4th input to regdst mux
- Control:
 - If stall_EX is asserted, don't write registers (no-op)
 - For branches:
 - Set aluop to **SLT** for blt, bge, **SLTU** for bltu, bgeu, and **SUB** for beq, bne
 - Use ALU output R_EX to resolve branches

Branch Resolution

Instruction	ALU op	Branch (pcsrc_EX=2' b01) if:
beq	sub	R_EX == 32'b0
bne	sub	R_EX != 32'b0
blt	slt	R_EX == 32'b1
bltu	sltu	R_EX == 32'b1
bge	slt	R_EX == 32'b0
bgeu	sltu	R_EX == 32'b0

Test Program

PC

```
0          li x4,-1
1          li x5,2
2          beq x4,x5,target3      # not taken
3          bne x4,x5,target3      # taken
4  target1: blt x4,x5,target4      # taken
5          jal target5
6          j exit
7  target2: nop
8  target3: jal x1,target1
9  target4: jalr x1
10 target5: bge x4,x5,target1      # not taken
11          bgeu x4,x5,target6     # taken
12          beq x0,x0,target1      # not executed
13 target6: bltu x4,x4,target1     # not taken
14  exit:
```

fetch sequence: 0,1,2,3,4(stall),8,9(stall),4,5(stall),9,10(stall),9,10(stall),10,11,12(stall),13

Lab 4: Branch/Jump Instructions

- Objectives:
 1. Implement RISC-V processor that implements instructions:
beq, bne, blt, bge, bltu, bgeu, jal, jalr
 2. Test with test program
 3. Write a MIPS test program:
 - Computes square root of a value on switches
 - Displays value on 7-segment LEDs
 - Implement on DE2
 - Value on switches is (18,0) value
 - Solution stored as (32,23) internally
 - Value displayed on HEXs is (8,5)
 - You can't light up decimal point on HEXs (not connected on DE2 board)

Fractional Conversion

- Switches allow a value from 0 to $(2^{18}-1)$
 - $0 \leq x \leq 262143$
 - $0 \leq x \cdot 5 < 512$
- Assume our BCD display is (8,5)
 - Need 3 digits to left of decimal point
 - Gives precision of $1/100000$ (least significant BCD value is 10^{-5})
 - Roughly equivalent to 2^{-14} ($2^{-14} = 6.1 \times 10^{-5}$)
- Binary representation: (32,14) fixed-point
 - shift initial step 14 bits to left
 - initial guess = 0, initial step = 256.0
- Use same algorithm as in lab 4, but first:
 - Multiply value by 10^5 (100000)
 - Shift 14 bits to the right